



INVENTORY MANAGEMENT

MR.PIBHU

MR.PANNAWISH

MR.AKKHRAWIN

CHITBURANACHART

KRIENGYAKUL

NAIR

**A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING**

**FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY**

2026

INVENTORY MANAGEMENT

MR.PIBHU	CHITBURANACHART
MR.PANNAWISH	KRIENGYAKUL
MR.AKKHRAWIN	NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL
FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE
BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

Computer Engineering Project
entitled
INVENTORY MANAGEMENT

Mr.Pibhu Chitburanachart
Researcher

Mr.Pannawish Kriengyakul
Researcher

Mr.Akkhrawin Nair
Researcher

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
Advisor

Computer Engineering Project
entitled
INVENTORY MANAGEMENT

was submitted to the Faculty of Engineering & International College,
Mahidol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 19, 2026

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
Committee Chair

Asst. Prof. Lalita Narupiyakul, Ph.D. (Computer Science)
Committee

Asst. Prof. Dr. Tuangyot Supeekit, Ph.D. (Industrial Engineering and Systems)
Committee

BIOGRAPHY

NAME	Mr.Pibhu Chitburanachart
DATE OF BIRTH	29 August 2004
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098-494-7984
E-MAIL	Pibhu22@gmail.com

NAME	Mr.Pannawish Kriengyakul
DATE OF BIRTH	17 April 2003
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098-970-5604
E-MAIL	pannawish.kri@gmail.com

BIOGRAPHY

NAME	Mr.Akkhrawin Nair
DATE OF BIRTH	22 September 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098-781-9644
E-MAIL	akkhrawin22@gmail.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to our advisor and committee members, Asst. Prof. Mingmanas Sivaraksa, Asst. Prof. Lalita Narupiyakul, and Asst. Prof. Dr. Tuangyot Supeekit. Their insight, advice, and knowledge were invaluable in helping us complete this project within the available time frame.

Besides our advisors, we would like to acknowledge the use of AI tools in this project. AI was used as a supporting tool for help, guidance, brainstorming, language refinement, and researching related ideas during the development and writing process. The final project decisions, implementation, testing, and submitted content remained the responsibility of the project team.

Finally, we would like to thank Mahidol University International College and the Faculty of Engineering, Mahidol University, for providing the academic environment and resources that supported this project.

Mr.Pibhu Chitburanachart

Mr.Pannawish Kriengyakul

Mr.Akkhrawin Nair

INVENTORY MANAGEMENT

STUDENTS	Mr. Pibhu Chitburanachart ICCI Mr. Pannawish Kriengyakul ICCI Mr. Akkhrwin Nair ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
DATE OF GRADUATION	July 19, 2026

ABSTRACT

This project presents an Inventory Management System for SME trading businesses that buy from suppliers and resell to customers. The system brings products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory checking, gross-margin visibility, dashboard summaries, and a read-only AI assistant into one web application.

The application was developed with React and Vite on the frontend, Django REST Framework on the backend, and MySQL as the database. The backend controls important rules such as stock calculation, sale stock validation, and eligibility for billing notes, payment batches, and credit notes.

The completed system improves stock and margin visibility and connects purchasing, sales, and financial document tracking. The AI assistant is limited to supported read-only questions about stock, partners, receivables, payables, and document references. When an OpenAI API key is configured, the model explains verified backend facts and adds a concise conclusion and highlights; otherwise, the system returns a deterministic local summary.

CONTENTS

	Page
BIOGRAPHY	i
ACKNOWLEDGEMENT	iii
ABSTRACT (ENGLISH)	iv
CONTENTS	v
LIST OF TABLES	viii
LIST OF FIGURES	x
CHAPTER 1 INTRODUCTION	1
1.1 Background	1
1.2 Objective	1
1.3 Scope	2
1.4 Expected Results	4
1.5 Timeline	4
CHAPTER 2 LITERATURE REVIEW	6
2.1 North American Industry Classification System: Wholesale Trade [5]	6
2.2 Qtier-Rapor: Managing Spreadsheet Systems and Improving Corporate Performance, Compliance and Governance [1]	6
2.3 Sole Traders Still Not Prepared for Digital Tax Returns in April [9]	7
2.4 A Full-Stack Web Solution for Online FMCG Management System Using MERN: Design, Implementation, and Evaluation [3]	7
2.5 Essentials of Inventory Management [4]	8
2.6 Inventory and Production Management in Supply Chains [7]	8
2.7 Inventory and Business Calculation Basis	8
2.7.1 Unit Conversion and Line Amounts	8
2.7.2 VAT Calculation	9
2.7.3 Current Stock	9
2.7.4 Stock Validation and Allocation	10

2.7.5	Inventory Dashboard Calculations	10
2.7.6	Purchase Payable and Payment Batch Calculations	11
2.7.7	Billing Note and Credit Note Calculations	12
2.7.8	Cashflow and Net Position	12
2.7.9	Quotation Validity	12
2.8	Aisha: A Custom AI Library Chatbot Using the ChatGPT API [2]	13
2.9	Summary	13
CHAPTER 3	METHODOLOGY	14
3.1	Development Approach	14
3.2	Requirement Areas and Web-App Pages	15
3.3	System Architecture	16
3.4	Frontend Methodology	18
3.5	Backend and API Methodology	18
3.6	Database Design Methodology	20
3.7	Business Workflow Design	21
3.8	Validation and Security Design	22
3.9	Testing Methodology	23
3.9.1	Automated Backend Validation Method	25
3.9.2	Calculation Validation Method	26
3.9.3	AI Chat Validation Method	26
3.9.4	AI Report Validation Method	27
3.9.5	Manual Scenario Test Method	28
3.9.6	Questionnaire Method	29
3.10	Tools and Technologies Used	30
CHAPTER 4	RESULTS	32
4.1	System Overview	32
4.2	Implemented Modules	32
4.3	System Implementation	33
4.4	Results by Workflow	44
4.5	Feature Evidence Summary	46
4.6	Testing and Evaluation Results	48

4.6.1	Automated Backend Validation	48
4.6.2	Calculation Validation	49
4.6.3	AI Chat Validation	51
4.6.4	AI Report Validation	53
4.6.5	Manual Scenario Test	54
4.6.6	Questionnaire Result	56
4.6.7	User Feedback Summary	57
4.7	Limitations Found During Development	58
CHAPTER 5	CONCLUSION	60
5.1	Brief Project Overview	60
5.2	Objective Summary	60
5.3	System Design Summary	61
5.4	Implementation Summary	61
5.5	Result Summary	61
5.6	Key Findings	62
5.7	Obstacles and Challenges	62
5.8	Future Work	62
REFERENCES		64
APPENDIX A	SOFTWARE USER GUIDE	66

LIST OF TABLES

Table	Page
1.1 Project Timeline	5
3.1 Requirement Areas and Implemented Pages	15
3.2 Record and Support Page Scope	16
3.3 Architecture Components	17
3.4 Main API Endpoint Groups	19
3.5 Primary Database Model Groups	20
3.6 Testing Summary	24
3.7 Automated Backend Test Method	25
3.8 Calculation Validation Method	26
3.9 AI Chat Validation Method	27
3.10 AI Report Validation Method	28
3.11 Manual Scenario Test Method	29
3.12 Questionnaire Method	30
3.13 Tools and Technologies	31
4.1 Workflow Results and Related Database Entities	45
4.2 Evidence of Implemented Features	47
4.3 Automated Validation Result Summary	49
4.4 Calculation Validation Input	50
4.5 Calculation Validation Output	51
4.6 AI Chat Validation Input and Output	52
4.7 AI Report Validation Input	53
4.8 AI Report Validation Output	54
4.9 Scenario Test Participant Summary	55
4.10 Average Scenario Test Time and Score	56
4.11 Questionnaire Result Summary	57
4.12 User Feedback Themes	58

A.1	Master Data Setup Checklist	71
A.2	Finance Follow-Up Modules	76
A.3	Quick Reference for Common User Goals	79

LIST OF FIGURES

Figure	Page
3.1 System Architecture Overview	17
3.2 Business Workflow Design	22
4.1 Dashboard Page	33
4.2 Inventory Page	34
4.3 AI Chat Assistant Page	35
4.4 AI Report Generation Page	36
4.5 Product Management Form	37
4.6 Purchase Workflow	38
4.7 Sales Workflow	39
4.8 Quotation Workflow	40
4.9 Billing Note Workflow	41
4.10 Credit Notes Page	41
4.11 Payment Batch Workflow	42
4.12 User Access Management Page	43
4.13 Activity Log Page	43
4.14 Settings Page	44
A.1 Main Sidebar Navigation	69
A.2 Product Form With Unit Conversion Setup	72
A.3 Quotation Entry With Stock Sufficiency Information	73
A.4 Purchase Detail With Receiving Status	74
A.5 Sales Detail With Delivery Status	75
A.6 Dashboard Overview for Business Review	77

CHAPTER 1

INTRODUCTION

1.1 Background

Small and medium-sized trading businesses often operate as intermediaries between suppliers and customers [5]. In daily work, the business has to buy products from suppliers, receive stock, sell products to customers, prepare documents, and check whether the sales price still gives a reasonable margin. Because of this, inventory management is not only about counting how many products are left in the warehouse. It also involves quotations, purchase orders, sales records, delivery status, receivables, payables, and stock availability.

Many small businesses still depend on spreadsheets, paper records, manual document folders, or general-purpose accounting tools to manage these tasks [1, 9]. This can work at the beginning, but it becomes harder when the business has more products, more suppliers, different product units, delayed purchases, cancelled items, customer billing, and supplier payments. It also becomes difficult to answer simple questions quickly, such as which products are low in stock, which sales can be billed, which purchases are ready for payment, or whether there is enough received stock to fulfill a sale.

This project was created to bring those daily tasks into one web application. The Inventory Management System focuses on the main workflow of a trading business: managing master data, recording purchases and sales, checking stock, preparing quotations, tracking billing notes and payment batches, handling credit notes, viewing dashboard summaries, and asking limited read-only questions through the AI assistant.

1.2 Objective

1. To develop a system that records purchase and sales transactions, including the current status of each process, with an attachment file function that links all supporting documents directly to the corresponding transactions.

2. To develop an inventory management system that displays the overall product stock in the warehouse and provides tools for calculating essential inventory information.
3. To integrate a read-only AI chatbot using the OpenAI API that explains backend-prepared facts when configured and retains a deterministic local summary when the API is unavailable.

1.3 Scope

The scope of this project is based on the features that are already implemented in the Inventory Management System codebase. The system covers the following areas:

1. **Master data management:** The system manages categories, products, suppliers, and customers. Product records include SKUs, previous SKUs, product names, categories, unit conversions, reorder levels, active or disabled status, image or PDF attachments, and supplier-specific sourcing information.
2. **Purchasing workflow:** The system records purchase transactions, supplier information, purchase line items, expected delivery dates, received dates, item statuses, VAT totals, bill discounts, supplier tax invoice references, uploaded documents, and payable totals. The backend keeps purchase status and purchase item status in sync.
3. **Sales workflow:** The system records sales transactions, customer information, customer purchase-order references, line items, delivery statuses, uploaded documents, VAT totals, discounts, billing-note links, credit-note links, and source quotation links. The backend checks stock before a sale can move into a stock-deducting status.
4. **Inventory and stock control:** Available stock is calculated from received purchase quantities minus committed sale quantities. Sale allocations use received purchase layers and support first-in, first-out allocation. The inventory page and dashboard show stock position, reorder information, demand, purchase pipeline, stock value, and product history.
5. **Quotation workflow:** The system stores quotations with customer and supplier references, validity information, shipping date, payment term, VAT mode, to-

tal amounts, line items, and supplier options. Quotations can also be linked to purchases and sales created from them.

6. **Finance workflow:** The system supports billing notes for customer receivables, payment batches for supplier payables, and credit notes for cancelled or returned sales lines. Backend eligibility endpoints prevent users from selecting invalid transactions for these finance documents.
7. **Dashboard, margin, and reporting:** The backend provides dashboard summary and segment endpoints for finance, trends, product movement, cashflow, order coverage, and gross-margin visibility. Product, quotation, and sales data retain the purchase-cost and selling-price information needed to review reseller margin.
8. **Authentication and user access flow:** The backend includes JWT login, refresh, and authenticated user profile endpoints. The frontend supports login, token refresh, logout, and guest mode. The system also includes a user-access page for managing users, roles, and role permissions. Backend-wide authentication enforcement is controlled by the `INVENTORY_REQUIRE_AUTH` setting, and protected API actions use Django model permissions when authentication enforcement is enabled.
9. **AI-assisted operation:** The system includes a text-based inventory assistant endpoint for read-only questions covering stock and fulfillment, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup. With an OpenAI API key, the model explains verified backend facts and returns a conclusion and question-specific highlights; without a key or after an API failure, the same endpoint returns its local summary. The system also includes an AI report page that generates printable business reports for a selected supplier, customer, or product.
10. **Language and settings:** The application supports English and Thai interface text. The Settings page allows the user to choose the interface language without changing the underlying business records.

1.4 Expected Results

The expected result of this project is a working web-based inventory management system that can record purchases and sales, track the status of each process, and attach supporting documents to the correct transaction. The system should also show stock levels based on received purchases and committed sales, so users can understand where the stock comes from and where it is used.

The system is also expected to make daily checking easier through dashboard summaries and the limited AI-assisted text interface. Users should be able to find supported information such as product stock, transaction references, receivables, payables, partner summaries, and stock exceptions without manually searching every page. Overall, the completed system should reduce manual checking, improve stock visibility, and support better daily decisions for an SME trading business.

1.5 Timeline

Table 1.1 shows the planned sequence of proposal, requirement, development, testing, and final-report work from November 2025 through July 2026.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews sources related to wholesale trading, business record keeping, web-based inventory systems, inventory planning, and domain-specific AI assistance. Each source-focused section explains the source's main contribution and identifies the concept that is relevant to the system requirements. Project-specific development methods and results are discussed later in Chapters 3 and 4.

2.1 North American Industry Classification System: Wholesale Trade [5]

The North American Industry Classification System defines wholesale trade as an intermediate stage in the distribution of merchandise. Wholesalers sell or arrange the purchase and sale of goods for resale, durable non-consumer goods, and raw or intermediate materials. This definition distinguishes a reseller from a manufacturer and shows that wholesale work connects an upstream supplier side with a downstream customer side.

The applicable concept is the two-sided trading flow. A suitable information system must connect products and stock with supplier purchases, customer sales, and the financial records created by those transactions. Treating purchasing and selling as parts of one flow makes it possible to follow both the movement of goods and the related money.

2.2 Qtier-Rapor: Managing Spreadsheet Systems and Improving Corporate Performance, Compliance and Governance [1]

Bishop discusses how organizations use spreadsheets to fill gaps in their main administrative systems. Spreadsheets are flexible and quick to create, but larger spreadsheet collections can be difficult to control because formulas, versions, ownership, and

review procedures may be inconsistent.

The applicable concept is controlled, centralized record keeping. Product, transaction, and finance data are easier to review when they share defined fields, validation rules, access controls, and a common database instead of being divided among unrelated spreadsheets. This concept supports traceability and reduces the risk of conflicting versions of the same business record.

2.3 Sole Traders Still Not Prepared for Digital Tax Returns in April [9]

The article reports Sage research on sole traders who still rely on spreadsheets, bank statements, and paper records for financial administration. It describes the continuing gap between manual record keeping and structured digital processes in small-business environments.

The applicable concept is gradual digital support for routine work. A small-business system should organize records without requiring users to understand complex accounting software. Clear workflows for stock, purchasing, sales, billing, and payment follow-up can make operational information easier to find while reducing repeated manual entry.

2.4 A Full-Stack Web Solution for Online FMCG Management System Using MERN: Design, Implementation, and Evaluation [3]

Mittal and Arora present a full-stack web solution for fast-moving consumer goods management. The described system combines inventory control, order processing, stock updates, billing, and user roles through a browser-based interface.

The applicable concepts are shared web access and integration between operational modules. A browser interface allows different staff roles to use the same current records, while a server-side application can coordinate stock changes, order status, billing, and access control. These concepts are relevant regardless of the specific frontend or backend framework used to implement them.

2.5 Essentials of Inventory Management [4]

Muller explains that inventory management is broader than counting the quantity physically available. It includes demand, replenishment, stock handling, inventory value, and the decisions required to balance availability with cost.

The applicable concepts are stock visibility, demand awareness, replenishment review, and inventory valuation. Useful inventory information should therefore show not only on-hand quantity, but also expected demand, incoming supply, reorder needs, average cost, stock value, and the transaction history behind those values.

2.6 Inventory and Production Management in Supply Chains [7]

Silver, Pyke, and Thomas describe inventory planning as a connected set of decisions involving demand, replenishment, lead time, service level, stockout risk, and cost. The text explains why an inventory position cannot be judged only from the quantity currently on hand.

The applicable concepts are demand during lead time, safety stock, reorder level, and order-up-to planning. Considering these values together provides a traceable basis for deciding when additional stock may be needed and how current demand and confirmed incoming supply affect that decision.

2.7 Inventory and Business Calculation Basis

The system applies the reviewed inventory concepts through a limited set of calculation groups. Equations in this section define quantity normalization, transaction totals, stock availability, replenishment indicators, and finance balances. The reorder calculations are based on demand, lead-time, safety-stock, and inventory-value concepts described by Muller and by Silver, Pyke, and Thomas [4, 7].

2.7.1 Unit Conversion and Line Amounts

Products may be purchased or sold in units that differ from their stock-counting unit. Equation 2.1 converts a transaction quantity into the product's base unit so that all stock movements use the same measure.

$$\text{Base Quantity} = \text{Quantity} \times \text{Conversion Factor} \quad (2.1)$$

Equation 2.2 calculates a line amount after one or more percentage discounts. Each discount percentage is limited to the range from 0% to 100%. For purchases, unit price represents unit cost; for sales and quotations, it represents selling price.

$$\text{Discounted Amount} = \text{Quantity} \times \text{Unit Price} \times \prod_{i=1}^n \left(1 - \frac{\text{Discount Percent}_i}{100} \right) \quad (2.2)$$

2.7.2 VAT Calculation

The general value-added tax rate used for taxable goods and services in Thailand is 7% [8]. Equation 2.3 shows how the same rate is applied when VAT is already included in an item total and when VAT must be added to an item total.

$$\begin{aligned} r &= 0.07, \\ \text{Included Before VAT} &= \frac{\text{Item Total}}{1 + r}, \\ \text{Included VAT} &= \text{Item Total} - \text{Included Before VAT}, \\ \text{Excluded VAT} &= \text{Item Total} \times r, \\ \text{Excluded Grand Total} &= \text{Item Total} + \text{Excluded VAT}. \end{aligned} \quad (2.3)$$

2.7.3 Current Stock

Stock is derived from completed stock movements rather than entered as an independent manual number. As shown in Equation 2.4, received purchase quantities add stock, while sale quantities in stock-committing statuses reduce it.

$$\begin{aligned} \text{Current Stock} &= \max(0, \text{Received Purchase Base Quantity} \\ &\quad - \text{Committed Sale Base Quantity}). \end{aligned} \quad (2.4)$$

Only purchase items marked as received contribute to available stock. Sale items reduce stock when they are packed, shipped, or delivered. Pending sales remain visible as demand but do not reduce current stock until they reach a stock-committing status.

2.7.4 Stock Validation and Allocation

Before inventory is issued for a sale, the requested quantity is compared with the quantity available. A request that exceeds available stock is rejected before the transaction can commit the stock movement.

When stock is available, the quantity is assigned to received purchase batches. The default method uses first-in, first-out allocation, so older available stock is used before newer stock. Equation 2.5 limits the quantity taken from a batch to the smaller of the remaining sale requirement and the quantity still available in that batch.

$$\text{Allocation Quantity} = \min(\text{Remaining Sale Quantity}, \text{Batch Available Quantity}) \quad (2.5)$$

Manual allocation may be used when a specific physical batch is selected. The selected batch must contain the same product, must already be received, and must have enough unallocated quantity.

2.7.5 Inventory Dashboard Calculations

The first calculation group prepares the cost and demand values used for reorder review. Equation 2.6 defines average unit cost, average daily demand, demand during lead time, safety stock, and the calculated reorder level.

$$\begin{aligned}
 \text{Average Unit Cost} &= \frac{\text{Received Purchase Value}}{\text{Received Purchase Units}}, \\
 \text{Average Daily Demand} &= \frac{\text{Sales History Units}}{\text{Sales History Days}}, \\
 \text{Lead-Time Demand} &= \text{Average Daily Demand} \times \text{Average Lead Time}, \\
 \text{Safety Stock} &= \text{Average Daily Demand} \times \text{Safety Stock Days}, \\
 \text{Reorder Level} &= \lceil \text{Lead-Time Demand} + \text{Safety Stock} \rceil.
 \end{aligned} \tag{2.6}$$

Equation 2.7 then uses the reorder values with pending demand, over-sold quantity, available stock, and pending purchases. It produces a recommended re-stock quantity and an estimated stock value for review.

$$\begin{aligned}
 \text{Order-up-to Level} &= \text{Reorder Level} + \lceil \text{Lead-Time Demand} \rceil, \\
 \text{Recommended Restock} &= \max(0, \text{Order-up-to Level} + \text{Pending Sales} \\
 &\quad + \text{Oversold Units} - \text{Available Stock} - \text{Pending Purchases}), \\
 \text{Stock Value} &= \text{Available Stock} \times \text{Average Unit Cost}.
 \end{aligned} \tag{2.7}$$

These indicators support reorder review rather than automatically placing an order. Supplier cost history can then be used to compare possible replenishment sources.

2.7.6 Purchase Payable and Payment Batch Calculations

Cancelled purchase lines should not remain payable, while the original purchase document should remain readable. Equation 2.8 adjusts the payable amount according to the value of non-cancelled lines and calculates a payment batch from the lines included in that batch.

$$\begin{aligned}
 \text{Payable Total} &= \text{Grand Total} \times \frac{\text{Non-cancelled Line Amounts}}{\text{All Line Amounts}}, \\
 \text{Payment Batch Total} &= \sum \text{Payment Batch Line Amount}.
 \end{aligned} \tag{2.8}$$

Unpaid payment lines can be updated when the related payable amount changes. Paid lines remain unchanged because they represent a completed payment record.

2.7.7 Billing Note and Credit Note Calculations

Credit notes reduce the amount still expected from a customer. Equation 2.9 calculates the net billing amount after active credit adjustments are deducted.

$$\text{Billing Net Amount} = \text{Billing Note Total} - \text{Active Credit Note Total} \quad (2.9)$$

2.7.8 Cashflow and Net Position

Open receivables and payables are grouped by due date for cashflow review. Equation 2.10 shows whether open customer receivables are greater or lower than open supplier payables.

$$\text{Open Net Position} = \text{Open Accounts Receivable} - \text{Open Accounts Payable} \quad (2.10)$$

An open item is overdue when its expected or planned payment date is earlier than the current date and it has not been completed.

2.7.9 Quotation Validity

Quotation validity is determined from the quotation date and the selected validity period. Calendar-day validity adds every day, while business-day validity skips weekends. A quotation may also be stored without a validity end date. This rule is described in text because it is a date-processing rule rather than an inventory calculation.

Together, Equations 2.1 to 2.10 provide the calculation basis for four purposes: consistent transaction quantities, correct document totals, controlled stock and replenishment review, and clear receivable or payable balances. Their implementation method is described in Chapter 3, and their validation results are reported in Chapter 4.

2.8 Aisha: A Custom AI Library Chatbot Using the ChatGPT API [2]

Lappalainen and Narayanan describe a custom chatbot for a university library. The chatbot uses prepared institutional information and a controlled application context so that answers remain focused on the library domain instead of becoming unrestricted general conversation.

The applicable concept is domain-limited question answering. An assistant connected to operational data should define the topics it supports, keep record-changing actions outside the chat interface, and clearly reject or redirect unsupported requests. This boundary is especially important when the available information includes stock or financial records.

OpenAI's structured-output guidance describes the use of a JSON Schema to constrain response fields [6]. The applicable concept is a predictable response contract: a system can require separate fields for an answer, a conclusion, and highlights while keeping factual values in application-prepared data.

2.9 Summary

The reviewed sources establish the main concepts used by the Inventory Management System. Wholesale-trade classification explains the connected supplier-to-customer flow. The spreadsheet and small-business sources support centralized records and straightforward digital workflows. The web-based FMCG study supports integrated browser-based modules and role-aware access.

The inventory-management books provide the basis for stock availability, lead-time demand, safety stock, reorder review, and inventory value. The chatbot and structured-output references support a limited read-only assistant with predictable response sections. These concepts are applied through the architecture and methods described in Chapter 3 and evaluated through the results in Chapter 4.

CHAPTER 3

METHODOLOGY

This chapter explains how the Inventory Management web application was developed. The focus is on the development approach, architecture, workflow design, validation method, testing method, and tools used in the project. In this project, we built the system as a React web application connected to a Django REST Framework backend and a MySQL database.

3.1 Development Approach

We developed the system step by step, starting from the data that every other workflow depends on. The first part was master data, such as products, categories, suppliers, and customers. After that, we developed the transaction modules: purchases, sales, and quotations. The finance-related modules, including billing notes, payment batches, and credit notes, were added after the main transactions were already working. Finally, we connected the dashboard, inventory workspace, AI pages, administration pages, and language settings to the backend-supported workflow.

The project was separated into three main layers:

1. **Frontend layer:** React and Vite were used to build the web pages, forms, filters, tables, modals, and tab navigation.
2. **Backend layer:** Django REST Framework was used to expose APIs, validate data, calculate stock, control eligibility rules, prepare dashboard and assistant responses, and support AI report generation when an API key is configured.
3. **Database layer:** MySQL was used through the Django ORM to store relational business records.

This structure helped keep the user interface separate from the business rules. The frontend guides the user, but the backend is still responsible for the important checks such as stock validation, transaction eligibility, and payable amount synchronization.

3.2 Requirement Areas and Web-App Pages

The web application is organized around the main work areas that a trading business needs each day. Each sidebar page has a specific purpose, and the pages are connected through shared product, supplier, customer, purchase, sale, and finance data.

Tables 3.1 and 3.2 describe how the business requirements are divided among the implemented pages. Table 3.1 covers the main operational pages, while Table 3.2 continues with record management, administration, and language support.

Table 3.1 Requirement Areas and Implemented Pages

Requirement Area	Web-App Page	Purpose in the System
Operation summary	Dashboard	Shows stock, finance, cashflow, product movement, and order coverage summaries.
Inventory control	Inventory	Shows stock health, reorder information, supplier options, product details, and quick purchase-order preparation.
Assistant support	AI Chat	Answers supported read-only questions from verified backend facts; configured AI responses add a conclusion and question-specific highlights.
Report generation	AI Report	Generates printable supplier, customer, or product reports with metrics, charts, record tables, and AI-written analysis.
Purchasing	Purchases	Records purchase orders, line items, receiving status, documents, VAT totals, discounts, and payable values.
Supplier payment	Payment Batches	Groups eligible purchases for supplier payment and tracks paid or unpaid lines.
Quotation	Quotation	Prepares quotations with line items, supplier options, validity dates, and conversion to purchase or sale records.
Sales	Sales	Records customer sales, stock allocation, delivery status, documents, VAT totals, and credit-note links.

Table 3.2 Record and Support Page Scope

Requirement Area	Web-App Page	Purpose in the System
Customer billing	Billing Notes	Groups eligible sales into customer billing notes and tracks received or outstanding lines.
Returns and cancellation adjustment	Credit Notes	Creates credit notes from eligible cancelled or returned sale items.
Product records	Products	Manages product details, SKUs, image/PDF attachments, units, reorder levels, active status, suppliers, margin information, and history.
Category records	Categories	Manages product categories using a parent-child category tree.
Supplier records	Supplier	Manages supplier profile, contact information, procurement contact, branches, shipping addresses, and terms.
Customer records	Customer	Manages customer profile, contact information, branches, shipping addresses, and billing terms.
Administration and audit	User Access / Activity Log	Manages users, roles, and permissions, and provides authorized review of system activity.
Language preference	Settings	Switches the application interface between English and Thai.

3.3 System Architecture

The system uses a three-tier web architecture. React is responsible for the user interface, Django REST Framework provides the API and business rules, and MySQL stores the relational records. The frontend does not connect directly to the database. Every important action goes through the API.

Figure 3.1 shows the direction of communication between the user interface, API, business services, database, and read-only assistant.

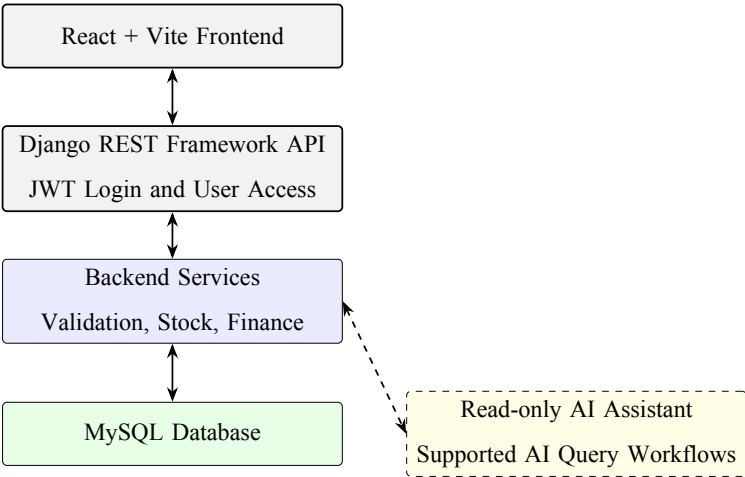


Figure 3.1 System Architecture Overview

Table 3.3 explains the responsibility and implementation used for each architecture layer shown in Figure 3.1.

Table 3.3 Architecture Components

Layer	Implementation	Main Responsibility
Frontend	React 18 and Vite	Builds the tabbed interface, forms, filters, tables, modals, and bilingual text display.
API	Django REST Framework	Receives requests, serializes data, applies filters, supports pagination, and returns JSON responses.
Business logic	Django service functions	Handles stock calculation, FIFO allocation, dashboard summaries, payable synchronization, eligibility checks, and assistant context.
Database	MySQL with Django ORM	Stores master data, transaction records, line items, documents, allocations, and finance records.
Authentication and access control	Simple JWT, Django groups, and Django permissions	Provides login, refresh token, authenticated profile support, role-based user access, and permission-gated API actions.

3.4 Frontend Methodology

The frontend was developed with React components, hooks, and helper files. Large pages are split into smaller sections, such as form details, line items, totals, directory tables, and detail modals. This made the code easier to manage because a purchase form, for example, has different concerns from a purchase history table.

The API client in `frontend/src/api.js` centralizes communication with the backend. It builds query strings, sends JSON or form-data requests, attaches bearer tokens, retries a request after token refresh, and dispatches an authentication-expired event when refresh fails. The frontend permission helper uses the profile returned from `/api/auth/me/` to show or hide navigation tabs such as products, purchases, sales, user access, and activity log. The data-loading hooks keep full lookup data separate from paginated directory rows, so forms can still access reference data while history pages can use pagination. User-facing text is selected from English and Thai dictionaries through the language context, and the Settings page controls the active interface language.

The frontend also keeps a mock-data fallback for development and demonstration. This fallback is useful when the backend is not running, but it is not treated as the final authority. The backend remains the source of truth for stock validation, eligibility, and financial synchronization.

3.5 Backend and API Methodology

The backend was developed as a Django project with one main inventory application. Django REST Framework viewsets provide CRUD endpoints for products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes. Separate API views provide dashboard summaries, dashboard segments, lookup lists, product history, product stock layers, eligibility lists, and chat responses.

Table 3.4 summarizes the main endpoint groups and the part of the application served by each group.

Table 3.4 Main API Endpoint Groups

Endpoint Group	Purpose
/api/products/, /api/categories/	Product catalog and category tree management.
/api/suppliers/, /api/customers/	Business partner master data management.
/api/purchases/, /api/sales/	Purchase and sales transaction workflows, including line items and documents.
/api/quotations/	Quotation records with normalized line items and supplier options.
/api/billing-notes/, /api/payment-batches/, /api/credit-notes/	Customer receivable, supplier payable, and credit note workflows.
/api/eligibility/...	Backend-controlled lists for billing-note sales, payment-batch purchases, and credit-note sale lines.
/api/dashboard/, /api/dashboard/segment/	Dashboard summary and segmented operational reports.
/api/products/<id>/history/ and /stock-layers/	Product transaction history and received stock layer details.
/api/chat/	Read-only assistant answers from backend-prepared facts. Configured OpenAI responses include a conclusion and highlights; failures use the local summary.
/api/ai-reports/	Generates printable HTML reports for a selected supplier, customer, or product with metrics, charts, and record tables.
/api/auth/login/, /refresh/, /me/	Login, token refresh, and authenticated user profile.
/api/admin/users/, /admin/roles/	User account management, role assignment, and role permission management for authorized administrators.
/api/activity-logs/	Read-only activity history for login, create, update, and delete events, available only to users with activity-log access.

Pagination is opt-in. If a page parameter is not sent, list endpoints can still return a plain array for compatibility with existing frontend logic. When pagination is requested, the response includes count, next, previous, page, page_size, total_pages, and results.

3.6 Database Design Methodology

The database was designed as a relational MySQL schema managed by Django migrations. Master data is separated from transaction data. For example, products, suppliers, customers, and categories are stored once and then referenced by purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system also keeps snapshot fields inside transaction records. A sale item stores product name, SKU, quantity, price, and amount. A purchase stores supplier name and line item details. These snapshot values are important because old business documents should remain readable even if a product, supplier, or customer is edited later.

Table 3.5 groups the principal database models according to the business records they store.

Table 3.5 Primary Database Model Groups

Group	Models
Master data	Category, Supplier, Customer, Product, ProductPicture, ProductUnitConversion, ProductSupplier
Purchasing	Purchase, PurchaseItem, PurchaseDocument
Sales	Sale, SaleItem, SaleItemAllocation, SaleDocument
Quotation	Quotation, QuotationItem, QuotationItemSupplier
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine

3.7 Business Workflow Design

The main workflow starts from master data. Products belong to categories and are used in purchase, sale, and quotation line items. Purchases bring stock into the system only when purchase items are received. Sales commit stock only when sale items are packed, shipped, or delivered. This is why the system does not treat product stock as a manually edited field.

Quotations are used before purchase or sales records are created. Billing notes are created from eligible sales for customer receivable tracking. Payment batches are created from eligible purchases for supplier payable tracking. Credit notes are created from eligible cancelled or returned sale lines. Dashboard summaries, AI Chat, and AI Report read from the operational records, while user access and activity logs support administration and audit review.

Figure 3.2 shows how master data, transactions, stock movement, finance documents, reports, and access control are connected in the overall process.

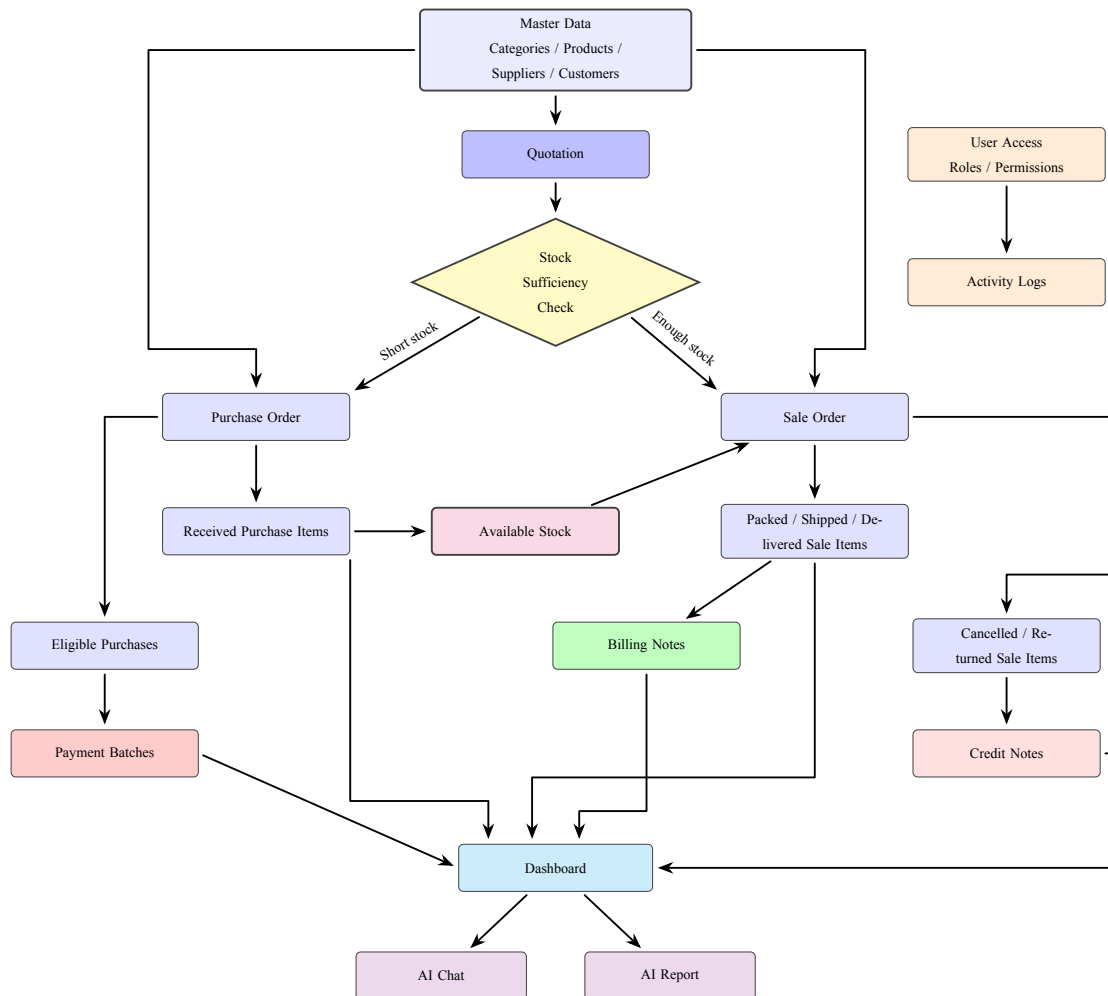


Figure 3.2 Business Workflow Design

3.8 Validation and Security Design

Validation is handled mainly by the backend. The frontend helps the user by showing form messages and previews, but direct API calls still need to be safe. The backend validates stock availability, active product usage, allocation rules, document eligibility, quotation validity dates, and finance document selection rules.

Security begins with the login process. A user enters an assigned username and password, and the backend verifies those credentials before issuing an access token and a refresh token. The frontend sends the access token with later API requests and uses the refresh token to obtain a replacement when the access token expires. After login, the authenticated profile endpoint returns the user's roles, permissions, and page-access

flags so that the interface and protected backend actions apply the same access rules. Simple JWT provides the tokens, while Django groups and model permissions define what each role can do. In local development, authentication can be optional through `INVENTORY_REQUIRE_AUTH`; in an authenticated deployment, protected actions require both a valid login and the necessary permission.

3.9 Testing Methodology

Testing was designed to check both technical correctness and user understanding of the program flow. The automated tests focused on backend behavior because the backend owns the most important business rules, including stock calculation, stock validation, finance eligibility, payable synchronization, and AI data preparation. Manual user testing followed the main transaction process, and questionnaires recorded whether participants could understand the interface and complete the required tasks.

Table 3.6 provides an overview of the areas checked before the individual testing methods are described.

Table 3.6 Testing Summary

Test Area	What It Checks
Pagination and filtering	Paginated responses, page size limits, search, status filters, date filters, and product or partner filters.
Product data	Product image/PDF upload, active/disabled behavior, lookup data, stock filter, average cost, margin visibility, and product history.
Stock validation	Sales cannot commit more stock than available received purchase layers.
Stock allocation	FIFO allocation, manual allocation, and stock layer remaining quantity are handled correctly.
Reorder calculation	Fixed mock purchase and sale data is used to confirm available stock, average cost, lead time, daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.
Finance eligibility	Billing notes, payment batches, and credit notes only use eligible records.
Financial synchronization	Purchase payable totals and payment batch line amounts stay consistent when purchases change.
User access control	Backend tests confirm that regular users cannot open administrator user endpoints, authorized administrators can create users and assign roles, and model permissions control protected API access when authentication enforcement is enabled.
Assistant scope	Supported questions return data-based answers, while unsupported questions are rejected as out of scope.
AI report output	Supplier, customer, and product report contexts are checked against backend data, and generated report HTML is sanitized and rendered with deterministic chart rows.

3.9.1 Automated Backend Validation Method

Automated backend validation was conducted with Django and Django REST Framework test cases. The tests used a temporary test database, created controlled records, called backend APIs or service functions, and compared the returned values with expected behavior. This method was used because backend rules must still be correct even if a user bypasses the frontend and calls the API directly.

The preserved black-box test history contains a selected 30-case workflow suite for presentation. That historical selection covered product validation, quotation behavior, purchase workflow, sales stock validation, finance eligibility, credit notes, payment batches, payable synchronization, and lookup behavior. It did not contain dedicated pagination or filtering cases. The final full backend suite contains 106 tests, including separate coverage for pagination, filtering, reorder calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging.

Table 3.7 describes how controlled data, test actions, and expected results were used for repeatable backend validation.

Table 3.7 Automated Backend Test Method

Method Item	Description
Test data	Controlled products, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes were created inside the test database.
Test action	The test cases called backend endpoints or backend service functions, similar to how the frontend sends requests during real use.
Expected output	The returned API data, status codes, calculated values, and validation errors were compared with the expected business rules.
Access-control checks	User-access tests checked that ordinary users cannot open administrator endpoints, administrators can create users and assign roles, permissions control protected API access, and activity-log access is restricted.
Reason for method	Automated tests are repeatable and suitable for checking exact rules such as stock validation, eligibility, and financial synchronization.

3.9.2 Calculation Validation Method

The calculation validation focused on the reorder point and inventory dashboard calculation. The method used fixed mock data instead of live random data so that the expected output could be manually calculated and compared with the backend result.

The test created one product, received purchase stock, committed sales, pending sales, pending purchases, and lead-time samples. After the records were created, the dashboard stock report calculation was executed. The output was checked against expected values such as available stock, average unit cost, average daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.

Table 3.8 summarizes the controlled inputs, comparison step, expected result, and reason for this calculation check.

Table 3.8 Calculation Validation Method

Method Item	Description
Input data	Fixed purchase and sale records were created for a known product so the expected stock calculation could be reproduced manually.
Test action	The backend dashboard stock report logic was called using the controlled test records.
Expected output	The backend should return the same reorder, restock, stock value, and stockout values as the manual calculation.
Reason for method	Reorder point output affects purchasing decisions, so the formula must be validated using traceable input data.

3.9.3 AI Chat Validation Method

The AI Chat check was designed to answer two simple questions: whether the assistant uses the correct system records and whether it stays within its read-only purpose. Known sample records were prepared, and the same questions were asked so that names, totals, stock values, and document links could be compared with expected answers.

The program was checked in two operating conditions. Without an OpenAI API key, it should still answer supported questions with a limited summary prepared by the system. With an API key, it may improve the wording and add a conclusion and highlights, but the facts must remain the same. Unsupported requests should receive a clear out-of-scope message and a list of the stock, partner, finance, or document questions that the assistant can answer.

Table 3.9 summarizes the sample information, questions, expected behavior, and purpose of this check.

Table 3.9 AI Chat Validation Method

Check	Description
Sample information	Known customer, sale, stock, billing, and payment records were prepared so their values could be checked directly.
Questions asked	The assistant received fixed questions about customer activity, net position, low stock, and a request outside its supported purpose.
Expected behavior	Supported questions should use the prepared records. The no-key condition should return a limited local summary, the configured-key condition should add an explanation, conclusion, and highlights, and unsupported questions should be declined.
Purpose	The check confirms that AI Chat helps users read existing information without inventing values or changing records.

3.9.4 AI Report Validation Method

The AI Report check examined whether a report showed the correct information for the supplier, customer, or product selected by the user. A small set of known records was prepared, a report was generated, and its names, transaction references, totals, charts, and tables were compared with those records. The print function and page structure were also checked.

The program was checked both with and without AI assistance. Without an OpenAI API key, the report should still contain the system-prepared facts and a local

written summary. With an API key, AI may add written analysis, but it must not replace the verified totals, records, or chart data. The check also confirmed that unsafe webpage content, such as scripts, was removed before the report was displayed.

Table 3.10 summarizes the information used, the report-generation step, the expected output, and the reason for the check.

Table 3.10 AI Report Validation Method

Check	Description
Sample information	A known supplier, customer, or product was selected together with related transactions, finance records, and stock information.
Report generation	The program created the report for the selected record and prepared it as a printable webpage.
Expected behavior	The report should show the correct record names, totals, references, charts, tables, and print control. Any unsafe webpage content should be removed.
Purpose	The check confirms that optional AI writing does not change the verified business values used in the report.

3.9.5 Manual Scenario Test Method

Manual scenario testing was conducted to check whether users could complete the main transaction process in the system. The scenario was divided into parts that followed the intended sequence: quotation, purchase creation, purchase receiving, supplier payment, sale checking, status update, credit note, and billing note.

During the test, each participant followed the scenario steps and recorded the time and score for each part. Tester names were removed from the stored result file for privacy. This method was used because automated tests can prove that rules work, but they cannot show whether users can understand the screen flow and find the required actions.

Table 3.11 describes the prepared tasks, the participant action, the expected outcome, and the reason for manual testing.

Table 3.11 Manual Scenario Test Method

Method Item	Description
Input data	Testers used the prepared system data and followed scenario tasks covering the main transaction and finance workflow.
Test action	Testers completed each scenario part while completion time, task score, and comments were collected.
Expected output	Users should be able to complete the main workflow and identify where the system is easy or difficult to use.
Reason for method	Manual testing checks practical usability and workflow clarity from the user's point of view.

3.9.6 Questionnaire Method

After the scenario test, participants answered a questionnaire using a five-point rating scale, where a higher value represented a more positive response. The questions covered ease of use, clarity of the transaction steps, usefulness of the information shown, and overall confidence in using the system.

The ratings were averaged to show the general response to each part of the system. Written comments were read and grouped by common ideas mentioned by more than one participant, such as difficulty finding buttons or understanding a status. The questionnaire supported the technical tests by showing how people understood the program after using it; it was not used as a replacement for functional validation.

Table 3.12 explains the information collected, how it was summarized, the expected result, and the purpose of the questionnaire.

Table 3.12 Questionnaire Method

Step	Description
Information collected	Participants provided ratings and optional written comments after completing the scenario tasks.
Summary method	Ratings were averaged, and comments with similar meaning were grouped into common positive points and improvement areas.
Expected result	The questionnaire should show how understandable and useful the system appeared to participants and identify interface areas that need improvement.
Purpose	The questionnaire adds user feedback that cannot be measured by automated program tests alone.

3.10 Tools and Technologies Used

The project used separate tools for frontend development, backend development, database storage, authentication, AI integration, styling, and testing. These tools were selected to support a full-stack web application with server-side business rules and a React-based user interface.

Table 3.13 lists each main tool and the role it serves in the completed system.

Table 3.13 Tools and Technologies

Category	Tool or Technology	Use in This Project
Frontend framework	React 18	Builds the web application screens and reusable UI components.
Frontend build tool	Vite	Runs the local development server and production build process.
Backend framework	Django 5	Provides project structure, ORM, migrations, settings, and backend application logic.
API framework	Django REST Framework	Provides serializers, viewsets, API views, JSON responses, parsers, and API tests.
Database	MySQL	Stores relational inventory, transaction, and finance data.
Authentication and access control	Simple JWT and Django permissions	Provides access and refresh token authentication, authenticated profiles, user-role management, permission checks, and activity-log access control.
AI integration	OpenAI API with local fallback	Uses structured model output for read-only chat explanations, conclusions, and highlights, and for AI report analysis when configured; backend-prepared local output remains available after missing keys or API failures.
Styling	CSS files by domain	Provides the compact inventory system interface.
Testing	Django TestCase and DRF APITestCase	Validates backend business rules and API behavior.

CHAPTER 4

RESULTS

This chapter explains the result of the implemented Inventory Management System. The result is based on the current codebase and implemented web-application pages. The calculation basis is defined in Chapter 2 and the development method in Chapter 3, so this chapter focuses on demonstrated workflows, evidence, and evaluation results.

4.1 System Overview

The final system is a full-stack web application for an SME trading business. It supports the flow from product setup to purchasing, selling, billing, supplier payment, credit note adjustment, inventory checking, dashboard reporting, and limited AI-assisted questions.

The main result is that the system connects business documents together instead of keeping them as separate records. A product can appear in purchases, sales, quotations, inventory reports, and product history. A sale can later become part of a billing note or a credit note. A purchase can later become part of a payment batch. This connection makes the system more useful than a simple product list.

4.2 Implemented Modules

All requirement areas mapped in Tables 3.1 and 3.2 were implemented. The completed workspace includes Dashboard, Inventory, AI Chat, AI Report, Purchases, Payment Batches, Quotation, Sales, Billing Notes, Credit Notes, Products, Categories, Supplier, Customer, User Access, Activity Log, and Settings. The following section presents page-level evidence without repeating the full requirement map.

4.3 System Implementation

This section combines the page-development explanation with selected screenshots from the finished application. Each figure is described and referenced first, with the corresponding screenshot placed directly underneath. This consistent order connects every screenshot to the text that explains it.

Figure 4.1 presents the dashboard page, which is the main post-login business overview. It receives backend dashboard data and shows stock health, reorder planning, stock cycling, delivery pipeline, purchase planning, popular products, finance, cashflow, and order coverage. It is connected to other pages, so a low-stock product can lead the user to the inventory or product page, and an order issue can lead to the purchase or sales page.

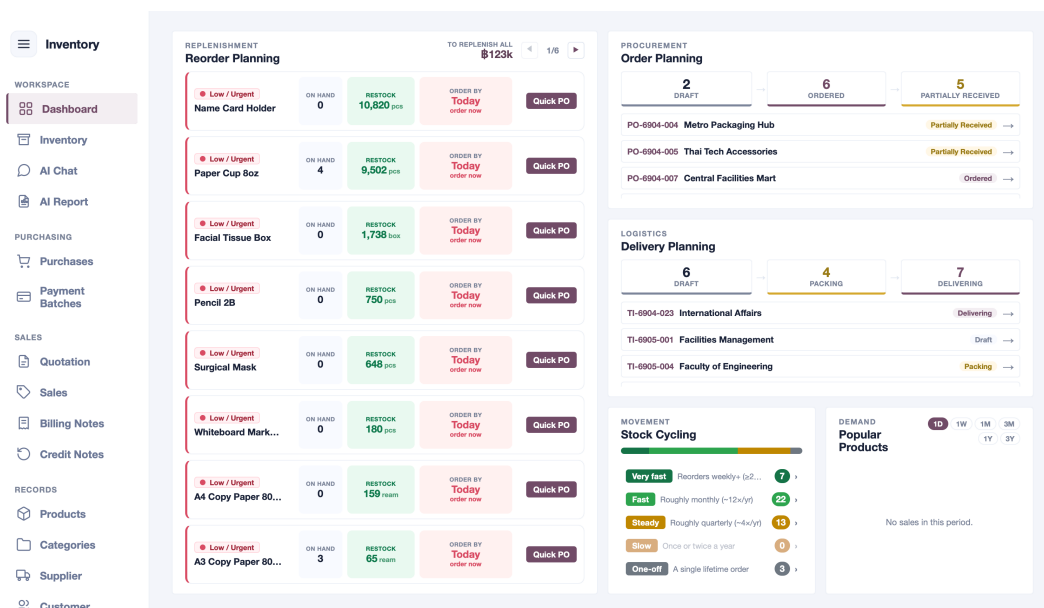


Figure 4.1 Dashboard Page

Figure 4.2 presents the inventory page used for daily stock checking and reorder review. It uses backend stock report rows instead of calculating stock from full transaction lists in the browser. The interface includes an inventory control board, searchable directory, filters for health status, category, supplier, value, and reorder need, product detail views, and a quick purchase-order drawer.

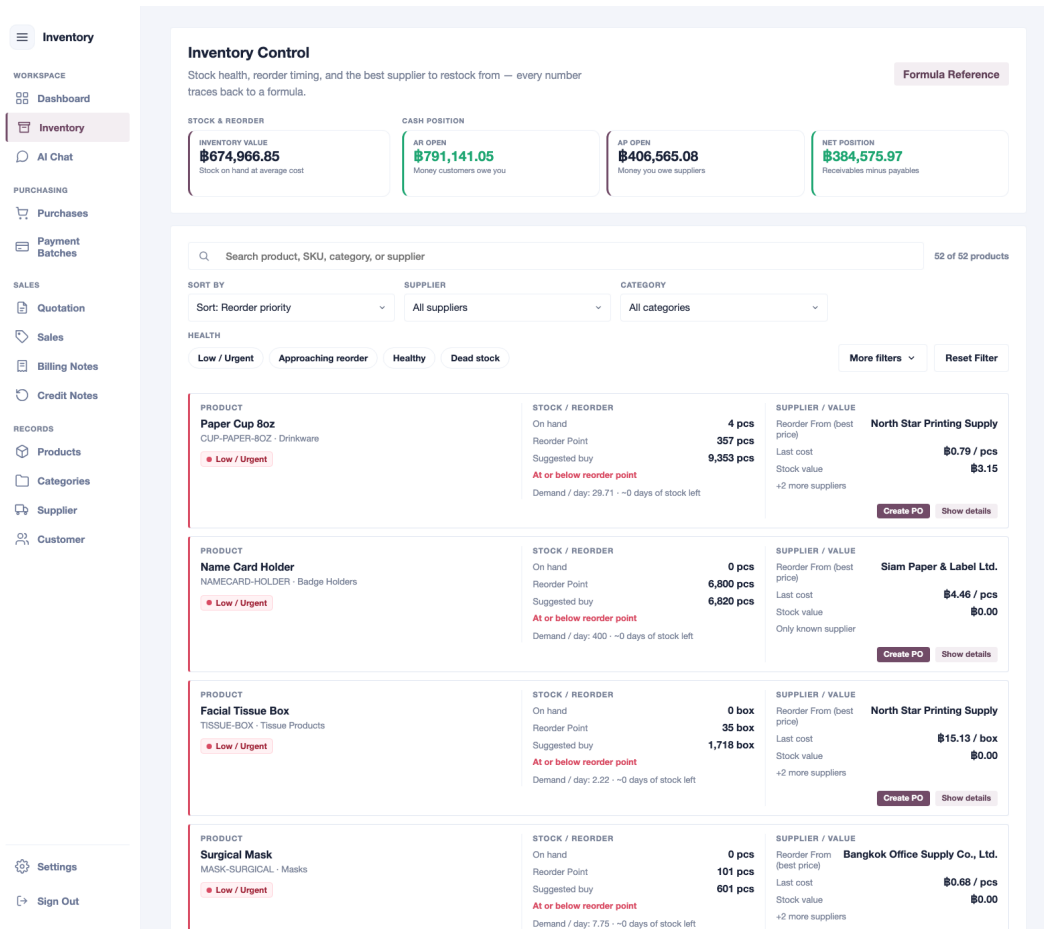


Figure 4.2 Inventory Page

Figure 4.3 presents the AI Chat page used for read-only questions about stock, partners, receivables, payables, and document references. The program first prepares the factual answer from current records. Without an OpenAI API key, the page displays a limited local summary. With a configured key, AI improves the explanation and adds a conclusion and question-specific highlights without changing the prepared facts. The assistant does not create, edit, approve, cancel, or delete records.

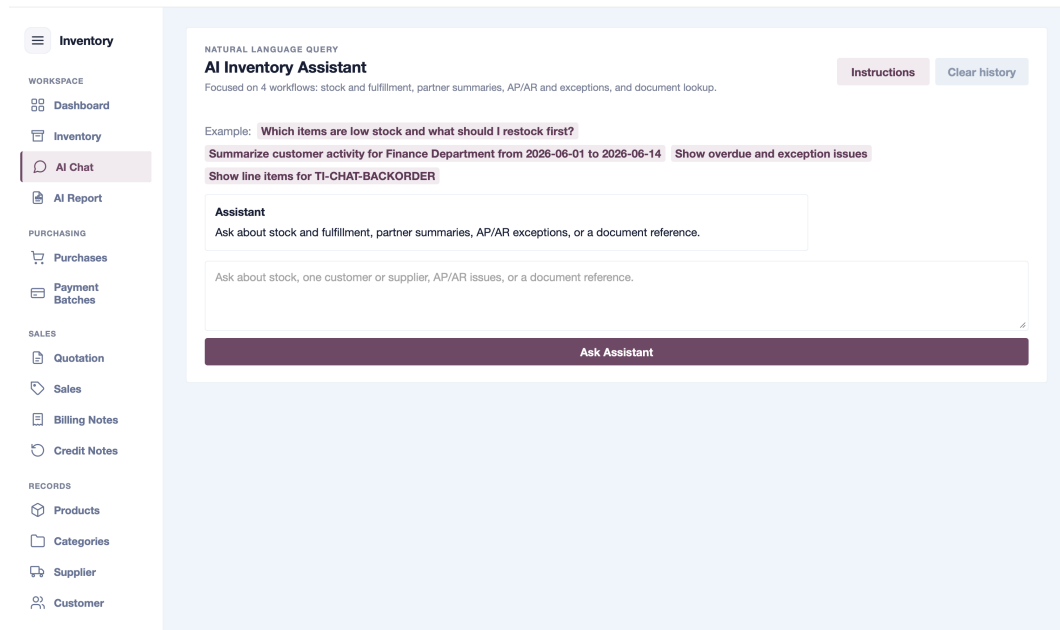


Figure 4.3 AI Chat Assistant Page

Figure 4.4 presents the AI Report page used to create printable supplier, customer, or product reports. The user chooses the report type and record and may narrow the reporting dates. The program collects related transactions, finance records, stock values, charts, and tables. Without an OpenAI API key, it produces a local report; with a configured key, AI adds written analysis to the same verified information. The final report opens as a printable webpage.

The screenshot displays the 'AI Report' generation interface. On the left, a sidebar menu lists various application sections: Inventory, Dashboard, AI Chat, AI Report (highlighted), Purchases, Payment Batches, Quotation, Sales, Billing Notes, Credit Notes, Products, Categories, Supplier, and Customer. The main panel is titled 'AI REPORT' and contains a form with three tabs: 'Supplier', 'Customer', and 'Product'. The 'Supplier' tab is active, showing a 'Supplier' dropdown menu with 'Bangkok Office Supply' selected, and a 'Time period' dropdown menu with 'All time' selected. Below these, a 'Report target' section shows 'Bangkok Office Supply' and 'All time'. At the bottom right of the form are two buttons: 'Reset' and 'Generate Report'.

Figure 4.4 AI Report Generation Page

Figure 4.5 presents the product form used to maintain item master data, including product identifiers, names, category, units and conversions, reorder settings, image or PDF attachments, supplier links, and cost or margin information. Products connected to transactions can be disabled instead of deleted so that historical purchases, sales, and quotations remain readable.

The screenshot shows a 'NEW PRODUCT' form in a software application. The form is titled 'Product product-1781862220030-oreawj'. It is divided into three main sections: 'IDENTITY', 'STOCK UNITS', and 'DETAILS'. The 'IDENTITY' section includes fields for 'Main Product Name', 'Category', and 'SKU'. The 'STOCK UNITS' section includes fields for 'Base Stock Unit', 'Default Purchase Unit', 'Default Sales Unit', and 'Unit Conversions'. The 'DETAILS' section includes a field for 'Picture of this product'. The form is displayed over a background showing a list of products and a sidebar with navigation options like 'Inventory', 'Purchases', 'Sales', and 'Records'.

Figure 4.5 Product Management Form

The other master-data pages support the same business foundation. The categories page organizes products in a parent-child tree and shows the full hierarchy in one view. The supplier page stores company profile, contact details, taxpayer ID, branches, shipping addresses, remarks, payment terms, and procurement contact details. The customer page stores company profile, contact details, branches, shipping addresses, remarks, and billing or payment term information.

Figure 4.6 presents the purchase page used to enter and review supplier purchase orders. It records the supplier, items, attached documents, overall purchase status, receiving status, taxes, discounts, and delivery dates. Only an item marked as received becomes available stock.

Inventory

WORKSPACE

Dashboard

Inventory

AI Chat

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

Settings

Sign Out

PURCHASE ENTRY

Add Purchase Transaction

Close

Reference No.

PO-6906-001

Supplier Name *

Search existing supplier

Payment Term

— Select payment term —

Supplier Tax Invoice

Optional supplier tax invoice

Status *

Ordered

Transaction Date *

19/06/2026

Payment Date

dd/mm/yyyy

Notes

Documents

No documents selected

Add Files

Attach invoices, receipts, or related files.

Purchase Items

Add Item

PRODUCT *	SKU	UNIT *	EXPECTED DELIVERY *
1 Search existing product	SKU	pcs	dd/mm/yyyy
QTY *	UNIT COST *	DISCOUNTS	AMOUNT
1	0.00	0 %	฿0.00

+ Add

ALL ITEMS DISCOUNT

Off

VAT SETTING

On

Include VAT

Exclude VAT

Total

฿0.00

VAT (7%)

฿0.00

Figure 4.6 Purchase Workflow

Figure 4.7 presents the sales page used to prepare and track customer orders. It records customer details, the customer purchase-order reference, items, documents, tax totals, discounts, and delivery statuses. Before a sale can reserve or issue stock, the available quantity is checked and the sold quantity is assigned to received purchase batches.

Inventory

WORKSPACE

Dashboard

Inventory

AI Chat

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

Settings

Sign Out

SALES ENTRY

Add Sale Transaction

Close

Reference No.
TI-6906-001

Customer Name *
Search existing customer

Payment Term
Select payment term

Status *
Draft

Transaction Date *
19/06/2026

Payment Date
dd/mm/yyyy

Customer PO Reference
Optional customer PO reference

Notes

Documents
No documents selected

Add Files

Attach invoices, receipts, or related files.

Sale Items

Add Item

	PRODUCT *	UNIT *	QTY *	UNIT PRICE *	AMOUNT	
1	Search existing product	pcs	1	0.00	฿0.00	Remove

DISCOUNTS
0 %
+ Add

STOCK SOURCE
Auto FIFO

UNIT COST
Optional

No stock sources available for FIFO preview yet.

ALL ITEMS DISCOUNT
Off

VAT SETTING
Include VAT Exclude VAT

On

Figure 4.7 Sales Workflow

Figure 4.8 presents the quotation page used before a quotation is converted into a purchase or sale. It records requested items, supplier cost options, selling prices, gross-margin review, validity, shipping date, payment terms, VAT, and total values. Quotation lines are stored in the relational *QuotationItem* and *QuotationItemSupplier* tables.

Inventory

WORKSPACE

Dashboard

Inventory

AI Chat

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

Settings

Sign Out

QUOTATION ENTRY

New Quotation

Cancel

Quotation Number

QT-000019

Quotation Date *

19/06/2026

Customer Name *

Search customer

Date of Shipping

dd/mm/yyyy

Valid Until Date *

30 days

Calendar days Business days No Valid Date

Expires: 19/07/26

Payment Term

— Select payment term —

Note

Quotation Items

Add Item

	PRODUCT *	UNIT *	QTY *	SALE AMOUNT
1	Search existing product	pcs	1	\$0.00
	SALE PRICE *	DISCOUNTS		
	0.00	0 %		
			+ Add	Remove
	SUPPLIERS (COST COMPARISON)			
	+ Add Supplier			

VAT SETTING

On

Include VAT

Exclude VAT

Total

\$0.00

VAT (7%)

\$0.00

Grand Total

\$0.00

Cancel

Create Quotation

Figure 4.8 Quotation Workflow

Figure 4.9 presents the billing note page used to group eligible customer sales into a receivable document. It tracks the included sales, outstanding or received status, expected and actual payment dates, bank references, and related credit notes.

Inventory

AI Chat

AI Report

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

ADMINISTRATION

User Access

Activity Log

BILLING NOTE

Create Billing Note

Cancel

Customer *

Billing Note Date

Search reference, customer, status, sale ref

14/07/2026

Expected Payment Date

Bank Reference

14/07/2026

Optional

Note

Optional

STEP 2

Choose Sales Orders

0 selected

Select a customer to see eligible sales.

STEP 3

Apply Credit Notes

0 credits selected

Select a customer to see eligible sales.

Total Billed

\$0.00

Less Credit Notes

\$0.00

Net Payable

\$0.00

Cancel

Create Billing Note

Figure 4.9 Billing Note Workflow

Figure 4.10 presents the Credit Notes page used to record adjustments after a sale is cancelled or returned. The backend provides eligible sale lines, and the credit note stores customer information, sale reference, optional billing note link, line items, and total credited amount. This keeps the credit document connected to the original sale item while preserving a readable adjustment record.

Inventory

AI Chat

AI Report

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

ADMINISTRATION

User Access

Activity Log

CREDIT NOTE

Create Credit Note

Cancel

Reference No.

Customer *

CN-6907-001

Search customer with cancelled or returned items

Credit Note Date

Sales Order

14/07/2026

Select a sale with cancelled or returned items

Apply to Billing Note (optional)

Not applied to a billing note

Note

Optional

STEP 2

Cancelled or Returned Items to Credit

0 selected

Select a customer to see eligible sales.

Subtotal

\$0.00

Total Credit

\$0.00

Cancel

Create Credit Note

Figure 4.10 Credit Notes Page

Figure 4.11 presents the payment batch page used to group eligible supplier purchases for payment. It stores the supplier, planned and actual payment dates, bank reference, total amount, and paid status for each included purchase.

PAYMENT BATCH

Create Payment Batch Cancel

Supplier * Batch Date

Planned Payment Date Bank Reference

Note

STEP 2

Choose Purchase Orders 0 selected

Select a supplier to see eligible purchases.

Total Amount \$0.00

Cancel Create Payment Batch

Figure 4.11 Payment Batch Workflow

Figure 4.12 presents the User Access page used by an administrator to manage staff accounts and roles. The login and access process uses password verification, access and refresh tokens, authenticated profiles, and permission checks to control protected pages and actions.

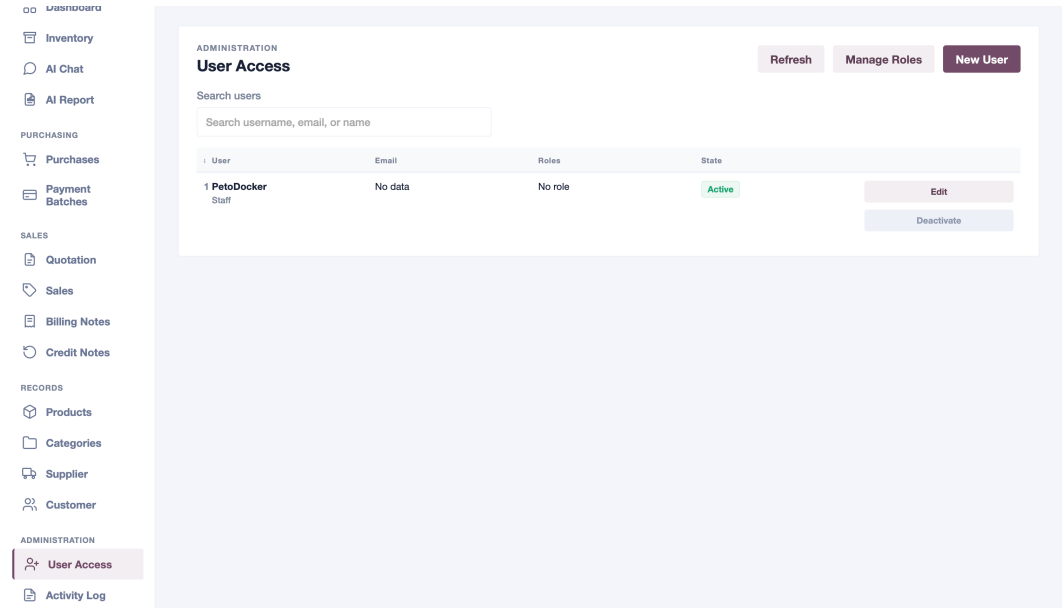


Figure 4.12 User Access Management Page

Figure 4.13 presents the Activity Log page used to review important events such as login, record creation, update, and deletion. Access to this page depends on the user's assigned role and permissions.

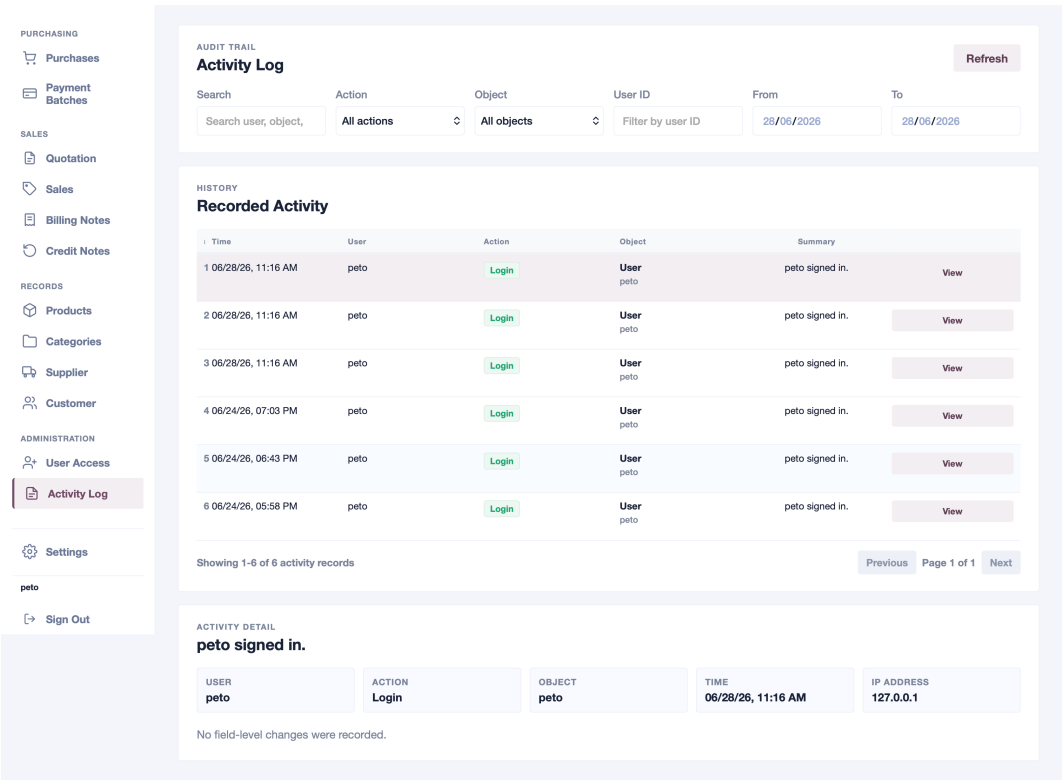


Figure 4.13 Activity Log Page

Figure 4.14 presents the Settings page used to switch the interface between English and Thai. The language choice changes user-facing labels and guidance while preserving the same records and backend business rules.

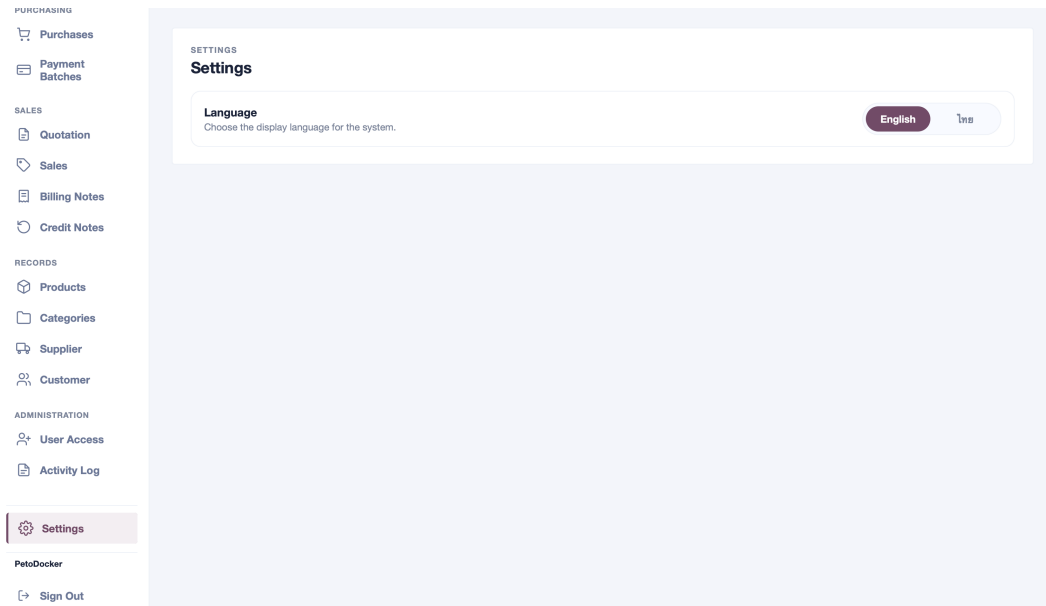


Figure 4.14 Settings Page

4.4 Results by Workflow

The workflow results are connected directly to the relational database design. Master records such as products, suppliers, customers, and categories provide the reusable business entities. Transaction records then reference those master records while keeping snapshot fields, so historical documents remain readable even if master data changes later. Line-item tables store the actual product quantities, prices, statuses, and totals used by stock, finance, dashboard, and report calculations.

Table 4.1 connects each visible workflow area to the main database records that produce its result. It shows that the pages are not isolated: master data supports transactions, transaction lines produce stock movement, and finance or reporting functions read those same records.

The key workflow result is that each module uses the same normalized database instead of keeping isolated screen-only data. Master data supports transactions, transaction line items drive stock and financial values, and finance documents reference eligible

Table 4.1 Workflow Results and Related Database Entities

Workflow Area	Main Database Entities	Relationship to the Result
Master data	Product, Category, Supplier, Customer, ProductPicture, ProductUnitConversion, ProductSupplier	These tables store reusable product and partner records. Transactions reference them through foreign keys while also keeping snapshot names, SKUs, and terms for historical documents.
Purchasing and stock	Purchase, PurchaseItem, PurchaseDocument, Product, Supplier	Purchase headers link to suppliers, and purchase items link to products. Received purchase items are the inbound stock source used by inventory and dashboard calculations.
Sales	Sale, SaleItem, SaleItemAllocation, PurchaseItem, Customer, Product	Sale headers link to customers, sale items link to products, and allocation rows connect sold quantities back to received purchase layers for traceability.
Quotations	Quotation, QuotationItem, QuotationItemSupplier, Customer, Supplier, Product	Quotation headers store customer and supplier context, quotation items store requested products, and supplier option rows store alternative costs for each line.
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine, Sale, Purchase	Finance documents group eligible sales or purchases through line tables, while credit notes reference the original sale so adjustments stay tied to the source transaction.
Dashboard and inventory	Product, PurchaseItem, SaleItem, ProductSupplier, BillingNote, PaymentBatch	Dashboard and inventory values are calculated from existing operational tables instead of storing a separate mutable stock total.
AI assistant and AI report	Product, Supplier, Customer, Purchase, Sale, Quotation, BillingNote, PaymentBatch, CreditNote	Assistant answers and printable reports use backend-prepared records from the same database entities that support the normal workflows.
User access and activity log	User, Group, Permission, ActivityLog	Django authentication tables store users, roles, and permissions, while the activity log records important system actions for review.

source records. This makes the system traceable from product setup through purchasing, sales, billing, payment, reporting, and audit review.

1. Master data provides the reusable product, category, supplier, and customer records needed by every transaction workflow.
2. Purchasing and sales line items are the main source for stock movement. Received purchase items add stock, while committed sale items allocate stock from received purchase layers.
3. Quotations, billing notes, payment batches, and credit notes keep business documents connected through database references, reducing duplicate or unrelated finance records.
4. Dashboard, inventory, AI chat, and AI report outputs use backend-prepared data from the same transaction tables, so summary information stays consistent with the stored records.
5. User access and activity logs add control and traceability by connecting staff accounts, roles, permissions, and important system events.

4.5 Feature Evidence Summary

Table 4.2 summarizes the implemented evidence used to confirm the main technical features. Each row states the observable behavior that demonstrates the feature in the completed system.

Table 4.2 Evidence of Implemented Features

Feature	Evidence in the Implemented System
REST API communication	The frontend API client calls endpoints for dashboard, lookups, products, purchases, sales, quotations, billing notes, payment batches, credit notes, authentication, and chat.
JWT authentication and user access	The backend provides login, refresh, profile, user-management, role-management, and activity-log endpoints. The frontend stores and refreshes access tokens, shows permission-based navigation, and provides a user-access page for authorized administrators.
Backend stock validation	The sale workflow calls backend validation before stock-deducting sale statuses are committed.
FIFO and manual allocation	The backend creates sale allocations from received purchase layers and validates manual layer selections.
Finance eligibility	Backend endpoints return only eligible sales for billing notes, eligible purchases for payment batches, and eligible sale lines for credit notes.
Normalized quotation items	Quotation lines and supplier options are stored in relational tables while the API still returns a simple items array to the frontend.
Backend dashboard summaries	The dashboard and inventory pages use backend-prepared summary data instead of calculating all operational metrics from full transaction lists in the browser.
Assistant scope control	The chat service answers only supported read-only workflows and rejects unsupported requests as outside scope.
AI report generation	The AI report endpoint collects transaction data for a selected record, calculates metrics and charts, and returns a printable HTML report with optional AI-written analysis.

The evidence shows that important rules are enforced by shared backend services and database records rather than by individual screens alone. This is especially important for stock allocation, finance eligibility, access control, and AI output because

these functions must remain consistent regardless of which page requests the information.

4.6 Testing and Evaluation Results

The system was evaluated using three types of evidence: automated backend validation, manual scenario testing, and user questionnaire results. Automated tests were used for business rules that must be exact, such as stock calculation, eligibility, and AI output validation. Scenario testing and questionnaires were used to evaluate how users performed the main workflow and how they perceived the system.

4.6.1 Automated Backend Validation

The final full backend suite contained 106 automated tests, and all 106 passed. This suite includes pagination, filtering, product data, transaction validation, stock allocation, finance eligibility, calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging. For presentation continuity, the Blackbook also preserves a historical 30-case black-box aligned workflow selection; all 30 selected cases passed. The smaller recorded 5-test AI/calculation run and 6-test access/activity run are therefore treated as earlier focused evidence rather than the current total.

Table 4.3 presents the test groups, pass results, and the main behavior confirmed by each group.

Table 4.3 Automated Validation Result Summary

Test Group	Result	Main Evidence
Full current backend suite	106/106 passed	The final Django inventory test suite passed across API, business-rule, calculation, AI, access-control, pagination, and filtering coverage.
Selected backend workflow tests	30/30 passed	Product rules, quotation behavior, purchase workflow, sale stock validation, finance eligibility, credit notes, payment batches, and payable synchronization passed.
Reorder point calculation	Passed	The backend returned reorder level 48, recommended restock 18, days until stockout 15, and stock value 300 from fixed mock data.
AI Chat validation	Passed	Local chat returned deterministic backend summaries; configured model responses retained the same presentation and added validated answer, conclusion, and highlights fields.
AI Report validation	Passed	Supplier, customer, and product reports used backend context, returned printable HTML, replaced poor model chart HTML with backend chart rows, and sanitized unsafe HTML.
User access and activity log validation	6/6 passed	Regular users were blocked from administrator endpoints, administrators could create users and assign roles, model permissions controlled protected access, and activity-log access was restricted.

4.6.2 Calculation Validation

The calculation validation focused on the reorder point and recommended restock calculation because this is one of the most important inventory decisions in the system. The test did not use random data. Instead, it created fixed purchase and sale records in a temporary Django test database so that the expected result could be calculated manually.

The test first created one product, two received purchase records, one pending purchase record, two committed sales, and one pending sale. Then the test called the dashboard API, which runs the same backend stock report calculation used by the real dashboard and inventory page. The backend output was compared with the manually calculated expected values.

Table 4.4 lists the controlled input values and explains why each value was included. Table 4.5 then compares the manually expected result with the value returned by the program. The tested calculation follows the reorder components and restock definitions in Equations 2.6 and 2.7.

Table 4.4 Calculation Validation Input

Input Item	Value	Reason for Test
Product	TEST-PEN	A single controlled product makes the stock row easy to identify.
Received purchase units	100 pcs	Used as the stock added into the system.
Received purchase value	500	Used to calculate average unit cost.
Committed sale units	40 pcs	Used as stock already allocated to fulfilled sales.
Pending sale units	20 pcs	Used as future demand that should affect restock recommendation.
Pending purchase units	10 pcs	Used as incoming stock that should reduce restock recommendation.
Lead time samples	4 days and 6 days	Used to calculate average lead time of 5 days.
Sales history	40 pcs over 10 days	Used to calculate average daily demand of 4 pcs per day.

Table 4.5 Calculation Validation Output

Output Item	Expected Value	Backend Value	Result
Available stock	60	60	Passed
Average unit cost	5	5	Passed
Average lead time	5 days	5 days	Passed
Average daily demand	4 pcs/day	4 pcs/day	Passed
Safety stock	28	28	Passed
Reorder level	48	48	Passed
Recommended restock	18	18	Passed
Days until stockout	15 days	15 days	Passed
Stock value	300	300	Passed

As shown in Table 4.5, every calculated output matched the manually expected value. The result confirms that the same controlled inputs produce available stock of 60 units, a reorder level of 48 units, a recommended restock quantity of 18 units, and a stock value of 300.

4.6.3 AI Chat Validation

The AI Chat validation checked whether the chat page answers from backend data instead of inventing business values. The test was performed by calling the backend chat function with fixed questions and known data. The important output fields were the answer text, presentation title, selected record references, model marker, conclusion, and highlights.

Without an API key, AI Chat returns a limited local summary prepared from the selected records. With a configured key, AI improves the wording and adds a conclusion and highlights while using the same factual information. If the AI request

fails, the local summary remains available. Requests outside the supported read-only topics receive an out-of-scope response instead of an invented answer.

Table 4.6 shows the sample questions, the known information used for comparison, the expected answer, and the result for both local and AI-assisted conditions.

Table 4.6 AI Chat Validation Input and Output

Test Question	Input Data Used	Expected Output Checked	Result
Customer activity summary	Customer with one known sale worth 275	Chat title shows the selected customer, sales count is 1, sales total is 275, and the sale record target points to the created sale.	Passed
Net position question	Existing billing note and payment batch data	Answer includes backend AR, AP, and net position values with presentation title ``Net position".	Passed
Low stock question	Product stock report rows from backend	Answer lists low-stock products and restock priorities from backend stock calculations.	Passed
Unsupported quotation summary	A broad request outside the supported assistant scope	Assistant returns ``Outside current assistant scope" and does not invent a quotation report.	Passed
Configured key condition	API key enabled with a mocked structured model response	Chat returns the configured model marker, model wording, conclusion, and highlights while retaining the backend presentation and record targets.	Passed

All checks in Table 4.6 passed. Customer totals, net position, and low-stock information matched the prepared records; the unsupported request was declined; and the configured-key condition added a conclusion and highlights without changing the verified record references.

4.6.4 AI Report Validation

The AI Report validation checked whether supplier, customer, and product reports are generated from the correct selected records. The test also checked that the final report is printable HTML, that chart rows come from backend data, and that unsafe model HTML is removed.

The report test used a controlled supplier and purchase record. The program prepared the supplier name, purchase reference, purchase amount, metrics, chart rows, and tables. For the configured-key condition, a fixed AI response was used so that the same expected result could be checked every time. This controlled approach keeps automated tests repeatable.

Table 4.7 lists the known report inputs. Table 4.8 lists the expected document, chart, content-safety, and current-data checks performed on the generated report.

Table 4.7 AI Report Validation Input

Input Item	Value	Purpose
Report scope	Supplier report	Confirms that the selected report type controls which records are collected.
Selected supplier	Configured Key Supplier	Confirms that the report context uses the selected supplier.
Product	Configured Key Product	Confirms that line-item data is included in the report context.
Purchase reference	PO-AI-KEY-001	Confirms that the report includes the expected transaction record.
Purchase amount	THB 450.00	Confirms that report totals and table values match backend data.
Model output	Simulated HTML body	Confirms how the system wraps model-style output into a final report document.

Table 4.8 AI Report Validation Output

Output Checked	Expected Result	Result
Report context	Contains Configured Key Supplier, PO-AI-KEY-001, and THB 450.00.	Passed
Printable document	Final HTML contains document structure and a print button.	Passed
AI model body handling	Simulated model summary remains in the final report after wrapping.	Passed
Chart rendering	Poor model chart labels are replaced with backend chart row labels.	Passed
HTML safety	Script tags, unsafe event handlers, iframes, and JavaScript links are removed.	Passed
Current-data report check	Supplier, customer, and product reports generated from current database records and contained expected references and totals.	Passed

All checks in Table 4.8 passed. The report contained the expected supplier, purchase reference, and amount; it remained printable; its charts used prepared data; and unsafe webpage elements were removed. These results show that optional AI writing did not replace the verified records and totals.

4.6.5 Manual Scenario Test

The manual scenario test asked users to complete the main process from quotation to purchase, purchase receiving, payment batch, sale, credit note, and billing note. Ten testers finished the scenario test.

Table 4.9 summarizes the participant group and scenario coverage. Table 4.10 then reports the average completion time, score, and interpretation for each part.

Table 4.9 Scenario Test Participant Summary

Item	Result
Number of participants	10 testers
Age range	19 to 68 years old
Average age	43.5 years old
Roles represented	SME-related users and general evaluators, including owner, manager, purchasing/sales staff, executives, and students.
Scenario parts	Part A to Part H, covering quotation, purchase, receiving, payment, sale, credit note, and billing note workflow.

Table 4.10 Average Scenario Test Time and Score

Part	Average Time	Average Score	Interpretation
Part A	151.9 seconds	9.0/10	Initial quotation workflow took the longest because it starts the process and requires more data entry.
Part B	71.8 seconds	8.9/10	Purchase creation from quotation was generally successful, but some users needed time to understand the flow.
Part C	29.7 seconds	9.5/10	Receiving purchase items was one of the faster and clearer tasks.
Part D	65.1 seconds	9.5/10	Payment-related workflow received high scores.
Part E	40.4 seconds	9.2/10	Sale-related checking was successful, although some users needed to read more information.
Part F	43.1 seconds	8.9/10	Status-related actions worked, but comments showed that status buttons should be clearer.
Part G	7.2 seconds	9.3/10	The shortest part, showing that the required action was quick once found.
Part H	49.1 seconds	9.4/10	Final billing or finance-related workflow received high scores.

The average scenario score across all parts was 9.21 out of 10. The lower relative scores in Part B and Part F match written comments about quotation-to-purchase flow, sale form details, and status-button clarity.

4.6.6 Questionnaire Result

After the scenario test, testers answered a 15-item questionnaire using a 1 to 5 rating scale. The average questionnaire score across all testers was 4.58 out of 5.

Table 4.11 summarizes the response count, scale, average score, and overall interpretation of the questionnaire.

Table 4.11 Questionnaire Result Summary

Evaluation Item	Result
Number of questionnaire responses	10 responses
Questionnaire items	15 items
Rating scale	1 to 5
Average questionnaire score	4.58/5
General result	Users gave mostly high scores, showing that the system was understandable and usable after completing the scenario.

4.6.7 User Feedback Summary

The user comments showed several repeated themes. Positive feedback mentioned that purchase order status was understandable, quotation-to-purchase flow was useful, credit note creation could be simple, and connected data reduced the need for repeated entry. Improvement feedback focused mainly on visual clarity and consistency.

Table 4.12 groups comments with similar meaning so that the most common positive points and improvement areas can be reviewed together.

Table 4.12 User Feedback Themes

Theme	Summary of Feedback
Easy areas	Users found purchase status, quotation creation, quotation-to-purchase flow, and connected data flow useful and understandable.
Difficult areas	Some users had difficulty finding buttons, especially save buttons, detail buttons, status buttons, and finance-related options such as billing notes or credit notes.
Visual clarity	Several comments suggested stronger colors, clearer status indicators, and more obvious emphasis for important actions or warning values.
Workflow consistency	Some users felt that clickable rows and save-button placement should be more consistent across pages.
Information density	Some users felt that certain pages, such as dashboard charts or cost information sections, contained many details and should be simplified or visually reorganized.

The recorded feedback identifies the main improvement areas as button visibility, status clarity, finance-action discoverability, and information density.

4.7 Limitations Found During Development

The system is functional, but some limitations remain:

- Backend-wide authentication is configurable, so the final deployment setting must be checked before production use.
- User access is implemented with role and permission management, but production deployment should still review the final staff roles, permission assignments, and administrator accounts before real company use.
- The frontend still keeps mock-data fallback behavior for demonstration, so final evaluation should use backend-connected data.
- Large-scale production performance testing with many users and a very large database was not included in the current codebase.

- User testing was conducted with 10 participants, so the result is useful for project evaluation but not enough to represent all possible SME users.

CHAPTER 5

CONCLUSION

5.1 Brief Project Overview

The Inventory Management System was developed as a web application for SME trading businesses that buy products from suppliers and resell them to customers. The system covers product and partner records, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory and margin review, dashboard summaries, limited AI-assisted questions, printable AI-generated reports, administration, activity logs, and language settings.

The project was built with a React and Vite frontend, a Django REST Framework backend, and a MySQL database. The main idea of the system is to connect stock, documents, and finance records together so the user can follow the business workflow from purchase to sale and payment.

5.2 Objective Summary

The project objectives focused on three main goals: recording purchase and sales transactions with status and document attachment support, showing useful inventory and margin information, and adding an AI chatbot that can answer supported questions from backend data.

The implemented system meets these goals within the current project scope. Purchases and sales support status tracking and documents. Inventory stock is calculated from received purchases and committed sales, while cost and selling-price data support gross-margin review. The assistant can answer supported read-only questions but cannot create or edit records; configured AI wording remains grounded in the backend-prepared presentation.

5.3 System Design Summary

The system was designed as a three-layer web application. The frontend handles the pages and user interaction. The backend handles APIs, validation, stock rules, eligibility rules, dashboard summaries, and assistant context. The database stores the actual business records using relational tables.

This design was useful because it keeps important business rules on the backend. The frontend can make the system easier to use, but the backend still controls the rules that protect stock, billing, supplier payments, and credit notes.

5.4 Implementation Summary

The frontend implementation includes pages for Dashboard, Inventory, AI Chat, AI Report, Purchases, Payment Batches, Quotation, Sales, Billing Notes, Credit Notes, Products, Categories, Supplier, Customer, User Access, Activity Log, and Settings. It also includes a shared API client, authentication handling, permission-based navigation, pagination support, filters, modals, document references, bilingual text, and mock-data fallback for development.

The backend implementation includes Django models, serializers, viewsets, service functions, dashboard endpoints, lookup endpoints, eligibility endpoints, authentication endpoints, user and role management endpoints, activity logging, AI report generation, management commands, and backend tests. The most important backend logic includes stock calculation, stock allocation, sale stock validation, margin inputs, purchase payable calculation, finance eligibility, permission enforcement, assistant fact preparation, structured AI response validation, and AI report context building.

5.5 Result Summary

The final result is a working inventory management system that connects the main trading workflows. A received purchase can increase stock, a committed sale can reduce stock, a billing note can track money expected from a customer, and a payment batch can track money that needs to be paid to a supplier.

The dashboard and inventory pages make the system easier to review because

they summarize stock and finance information in one place. The assistant adds another way to ask about supported operational information while staying within a narrow read-only scope. When configured, OpenAI adds an explanation, conclusion, and highlights to verified backend facts; local fallback remains available. The AI Report page generates printable supplier, customer, or product reports with metrics, charts, and business analysis.

The quantitative evaluation was consistently positive. The final automated backend suite passed 106 of 106 tests, while the preserved presentation subset passed 30 of 30 cases. Ten scenario-test participants achieved an average score of 9.21 out of 10, and the questionnaire average was 4.58 out of 5. These results show strong technical correctness and favorable user evaluation within the tested scope.

5.6 Key Findings

The main finding is that the system is most useful when inventory, margin, sales, purchasing, billing, payment, and reporting are connected through the same transaction records. This makes stock status, reseller margin, financial follow-up, and business documents easier to check from one system instead of being managed separately. The evaluation also indicates that the main remaining usability issues are button visibility, status clarity, finance-action discoverability, and information density rather than missing core workflows.

5.7 Obstacles and Challenges

The main challenge was keeping connected workflows consistent. Purchase, sale, stock, billing, payment, and credit-note records affect one another, so the backend had to enforce rules while the interface still stayed practical for daily users.

5.8 Future Work

Future work should focus on production readiness and usability:

1. Test the system with a larger SME user group and larger operational database.
2. Review production roles, permissions, and deployment settings with a real busi-

ness.

3. Improve audit details and visual clarity for important workflow status changes.

REFERENCES

- [1] K. Bishop. Qtier-Rapor: Managing spreadsheet systems and improving corporate performance, compliance and governance, 2008. URL <https://arxiv.org/abs/0803.0011>.
- [2] Y. Lappalainen and N. Narayanan. Aisha: A custom ai library chatbot using the ChatGPT api. *Journal of Web Librarianship*, 17(3):37--58, 2023. doi: 10.1080/19322909.2023.2221477.
- [3] H. Mittal and M. Arora. A Full-Stack Web Solution for Online FMCG (Fast-Moving Consumer Goods) Management System Using MERN: Design, Implementation and Evaluation. *International Journal of Latest Technology in Engineering, Management & Applied Science*, 14(4):1089--1094, 2025. doi: 10.51583/IJLTEMAS.2025.140400132.
- [4] M. Muller. *Essentials of Inventory Management*. AMACOM, New York, NY, 2 edition, 2011. ISBN 9780814416556.
- [5] Office of Management and Budget. *North American Industry Classification System: United States, 2022*. Executive Office of the President, Office of Management and Budget, 2022. URL https://www.census.gov/naics/reference_files_tools/2022_NAICS_Manual.pdf. Sector 42, Wholesale Trade.
- [6] OpenAI. *Structured Model Outputs*. OpenAI Developer Documentation, 2026. URL <https://developers.openai.com/api/docs/guides/structured-outputs>. Accessed 14 July 2026.
- [7] E. A. Silver, D. F. Pyke, and D. J. Thomas. *Inventory and Production Management in Supply Chains*. CRC Press, Boca Raton, FL, 4 edition, 2016. ISBN 9781466558618.
- [8] The Revenue Department of Thailand. *Value Added Tax*. The Revenue Department of Thailand, 2026. URL <https://www.rd.go.th/english/6043.html>. General VAT rate of 7 percent; accessed 14 July 2026.
- [9] The Times. Sole traders still not prepared for digital tax returns in april. The Times, December 2025. Reports Sage research on sole traders' use of spreadsheets, bank statements, and pen-and-paper accounting.

APPENDICES

APPENDIX A.

SOFTWARE USER GUIDE

This appendix provides a practical guide for using the Inventory Management web application. It is written for daily users such as administrators, product controllers, purchasing staff, sales staff, and finance staff. The guide follows the same business workflow used by the system: prepare master data, create quotations, buy from suppliers, sell to customers, follow up billing and payments, and review the business situation.

A.1 Starting the Software

The software can be started in two main ways. The recommended approach for demonstration is Docker Compose because it starts the MySQL database, Django backend API, and React/Vite frontend together. Manual startup can also be used when the developer wants to run each service directly on the local machine.

A.1.1 Starting with Docker Compose

Before starting, Docker Desktop should be installed and running. From the project root directory, run the following command:

```
docker compose up --build
```

The first startup may take a few minutes because Docker needs to download images and build the application containers. When startup is complete, open the frontend application in a browser:

```
http://127.0.0.1:5173/
```

The backend API and Django admin are available at:

```
http://127.0.0.1:8000/api/
```

```
http://127.0.0.1:8000/admin/
```


The backend container runs database migrations automatically before the server starts. The MySQL container uses port 3306 inside Docker and is exposed to the local machine on port 3307 to avoid conflict with a local MySQL server.

To add demonstration operational data after the containers are running, use:

```
docker compose exec backend python manage.py seed_operational_data
```

To create an administrator account for the Django admin page, use:

```
docker compose exec backend python manage.py createsuperuser
```

To stop the running containers while keeping the database volume, use:

```
docker compose down
```

To reset the Docker database completely, use the following commands only when the existing Docker database data should be deleted:

```
docker compose down -v
```

```
docker compose up --build
```

If AI Chat and AI Report should use live AI generation, start Docker Compose with an OpenAI API key:

```
OPENAI_API_KEY=your-key docker compose up --build
```

A.1.2 Starting Manually without Docker

Manual startup requires a local MySQL server, Python, and Node.js. First, create the MySQL database and user expected by the default backend environment file:

```
CREATE DATABASE inventory_db
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_unicode_ci;
```

```
CREATE USER 'inventory_user'@'localhost'
IDENTIFIED BY 'inventory_password';
GRANT ALL PRIVILEGES ON inventory_db.* TO 'inventory_user'@'localhost';
GRANT ALL PRIVILEGES ON test_inventory_db.* TO 'inventory_user'@'localhost';
FLUSH PRIVILEGES;
```

In the first terminal, start the backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

In the second terminal, start the frontend:

```
cd frontend
npm install
npm run dev -- --host 127.0.0.1
```

Then open the application at:

<http://127.0.0.1:5173/>

With the backend virtual environment active inside the backend directory, demonstration data and an administrator account can be created with:

```
python manage.py seed_operational_data
python manage.py createsuperuser
```

A.2 Accessing the System

Users access the system through the web application sign-in page. A staff member should sign in with the username and password assigned by the administrator. If the system is used for demonstration or training only, the guest mode can be used to open the application with mock data. Guest mode should not be used for real business records because changes are not saved to the live database.

Figure A.1 shows the sidebar used after sign-in. It groups the operational, transaction, record, administration, and Settings pages so that users can move between related tasks.

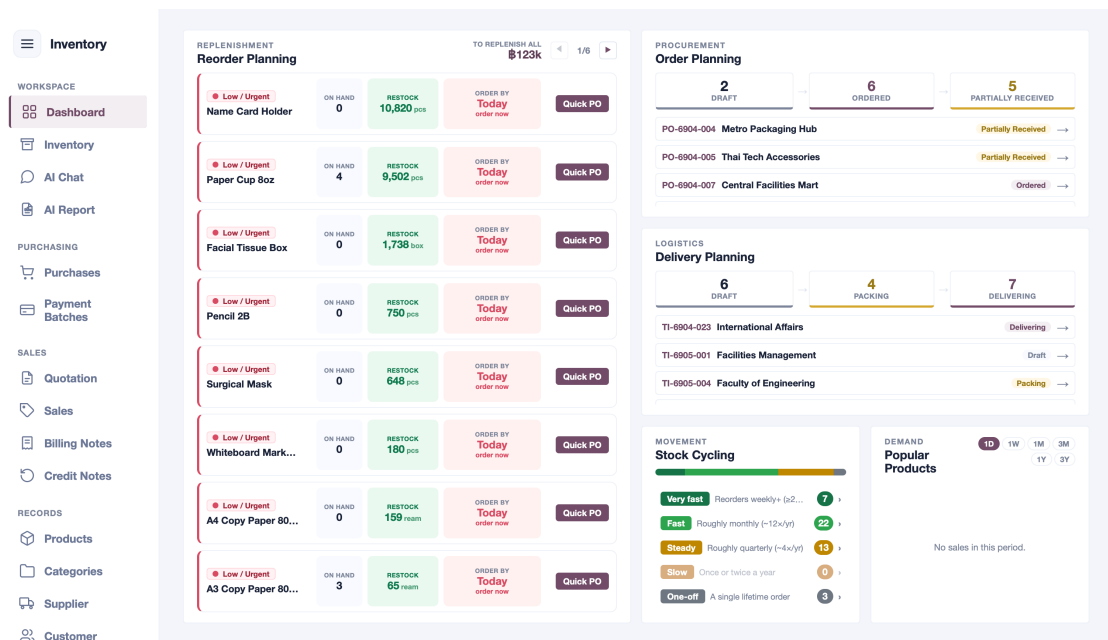


Figure A.1 Main Sidebar Navigation

The workspace group contains Dashboard, Inventory, AI Chat, and AI Report. The purchasing group contains Purchases and Payment Batches. The sales group contains Quotation, Sales, Billing Notes, and Credit Notes. The records group contains Products, Categories, Supplier, and Customer. Administration pages such as User Access and Activity Log are shown only to users with suitable permissions, while Settings provides the English/Thai language choice.

A.3 Basic Operating Rules

Before using the system, users should understand the following rules because they affect stock, cost, and financial records.

1. Product stock is calculated from transactions. Users do not type a free stock number into the product record.
2. Purchase items increase available stock only after the item status is changed to received.
3. Sales items reserve or consume stock when they move into stock-affecting statuses such as packed, shipped, or delivered.
4. The system uses first-in, first-out allocation by default, but users may choose manual stock layers when a specific received batch is physically used.
5. Historical transactions keep snapshot information such as product names, partner names, prices, tax values, and totals. Later edits to master data should not rewrite old business documents.
6. Billing notes, payment batches, and credit notes should be created from eligible records shown by the system instead of manually choosing unrelated transactions.

A.4 First-Time Master Data Setup

The first step is to prepare the records that transactions depend on. Categories should be created as a clear hierarchy. Products should then be linked to the correct category and configured with SKU, base unit, purchase unit, sales unit, unit conversions, and reorder information. Suppliers and customers should also be created before purchase and sales transactions are entered.

Table A.1 lists the minimum information that should be prepared in each master-data module before transactions are entered.

Table A.1 Master Data Setup Checklist

Module	Required Setup
Categories	Create the parent and child category tree used to organize products.
Products	Enter SKU, product name, category, base unit, purchase and sales units, unit conversions, reorder point, active status, and any image or PDF attachment.
Suppliers	Enter supplier name, tax ID, contacts, address, payment terms, and bank or procurement details.
Customers	Enter customer name, tax ID, billing address, shipping address, contacts, and payment terms.

Figure A.2 shows where product units and conversion quantities are entered. The base unit should represent the smallest unit used for stock calculation. For example, if a product is bought by box but sold by piece, the base unit may be piece and the purchase unit may be box. A conversion such as one box equals ten pieces allows purchase quantities to be converted into stock quantities correctly.

A.5 Daily Transaction Workflow

The normal daily workflow begins with a quotation or a direct purchase or sale. A quotation records customer demand and checks whether the requested items have enough available stock. If the quotation shows that stock is short, the user can prepare a purchase from the short-stock lines. If stock is available, the user can convert the quotation into a sale.

Figure A.3 shows the quotation entry screen and the stock-sufficiency information used before conversion.

Purchases record supplier-side orders. Users choose the supplier, add products, quantities, units, costs, and expected delivery dates, then save the purchase. When goods arrive, each purchase item should be updated to received only after the physical stock has been checked. This receiving action is what makes stock available for future sales.

Figure A.4 shows the purchase detail and the receiving status that controls when

Inventory

WORKSPACE

Dashboard

Inventory

AI Chat

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

Settings

PRODUCTS

Find

S

Filter

QUICK F

HISTORY

Produ

#

1

2

3

4

5

6

7

8

9

10

11

12

13

14

A3 Copy Paper 80gsm

SKU A3-80Q-RM

A3 Copy Paper

A3 Copy Paper 80gsm used for purchasing, stock, sal...

3 ream

Buy carton · Sell ream

\$214.16

NEW PRODUCT

Product product-1779920456475-i550ez

X

IDENTITY

Name, Category, and SKU

Start here

Main Product Name *

Name used across the system

Category *

Search category

SKU *

e.g. 0102030001

Generate

Required and unique. Generate uses category path codes plus a 4-digit serial.

SUB NAMES

Add alternate names and promote any one of them to the main name.

Add Sub Name

No sub names added yet.

STOCK UNITS

Base Unit and Conversions

Inventory logic

Base Stock Unit *

pcs

Default Purchase Unit *

pcs

Default Sales Unit *

pcs

UNIT CONVERSIONS

Factor means how many base units are inside one selected unit.

Add Unit

Unit *

pcs

Factor to Base *

1

Purchase

Sale

X

DETAILS

Image and Notes

Optional detail

Picture of this product

No pictures attached

Add Pictures

No picture selected.

Cancel

Save Product

Figure A.2 Product Form With Unit Conversion Setup

Inventory

WORKSPACE

Dashboard

Inventory

AI Chat

PURCHASING

Purchases

Payment Batches

SALES

Quotation

Sales

Billing Notes

Credit Notes

RECORDS

Products

Categories

Supplier

Customer

Settings

QUOTATION ENTRY

New Quotation

Cancel

Quotation Number

QT-000019

Quotation Date *

28/05/2026

Valid Until Date *

30 days

Calendar days

Business days

No Valid Date

Expires: 27/06/26

Customer Name *

Search customer

Note

Quotation Items

Add Item

PRODUCT *

1

Search existing product

UNIT *

pcs

QTY *

1

SALE AMOUNT

\$0.00

SALE PRICE *

0.00

DISCOUNTS

0

%

+ Add

Remove

SUPPLIERS (COST COMPARISON)

+ Add Supplier

VAT SETTING

Include VAT

Exclude VAT

On

Total

\$0.00

VAT (7%)

\$0.00

Grand Total

\$0.00

Cancel

Create Quotation

Figure A.3 Quotation Entry With Stock Sufficiency Information

purchased units enter available stock.

PURCHASE DETAIL
PO-6905-013

SUPPLIER
Premier Toner House

TRANSACTION DATE
25/05/26

PAYMENT DATE
24/06/26

SUPPLIER'S TAX INVOICE
4303-2605-0072

PAYMENT BATCH
-

NOTES
Cancelled after supplier could not meet delivery schedule. Includes converted units.

ITEMS

#	Product	SKU	Expected Delivery	Lead Time	Item Status	Received Date	Qty	Base Qty	Unit Cost	Base Cost Before Disc.	Base Cost After Disc.	Discounts	Amount	
1	Cyan Printer Ink Bottle	INK-CYN-001	25/05/26	-	Cancelled	dd/mm/yy	4 case	48 bottle	\$1,754.98	\$146.25	\$143.32	ITEM 2% BILL -	\$6,879.52	Mark Received
2	Facial Tissue Box	TISSUE-BOX	25/05/26	-	Cancelled	dd/mm/yy	6 box	6 box	\$16.23	\$16.23	\$15.42	ITEM 5% BILL -	\$92.51	Mark Received
3	USB-C Power Adapter 65W	ADP-USB65W	25/05/26	-	Cancelled	dd/mm/yy	1 carton	20 pcs	\$8,516.15	\$425.81	\$396.51	ITEM 3%4% BILL -	\$7,930.24	Mark Received
4	Blue Ballpoint Pen 0.5mm	PEN-BL-05	25/05/26	-	Cancelled	dd/mm/yy	8 box	400 pcs	\$281.73	\$5.63	\$5.63	ITEM - BILL -	\$2,253.64	Mark Received
5	Reflective Safety Vest	VEST-SAFETY	25/05/26	-	Cancelled	dd/mm/yy	8 carton	240 pcs	\$2,688.88	\$89.63	\$86.94	ITEM 3% BILL -	\$20,865.71	Mark Received

Original total	\$38,021.82
Cancelled (5 items)	-\$38,021.82
Amount payable	\$0.00

RECORDS

ITEM RECEIVING STATUS	0 Pending	0 Delayed	0 Received	5 Cancelled				
13	PO-6905-001	Bangkok Office Supply C...	Received	02/05/26	5	1181-2605-0060	\$22,876.27	Details
14	PO-6904-015	Warehouse Direct Thailand	Ordered	30/04/26	3	6525-2604-0059	\$19,555.58	Details

Figure A.4 Purchase Detail With Receiving Status

Sales record customer-side orders. Users choose the customer, add products, quantities, prices, and delivery-related information. The system checks stock and creates stock allocations from received purchase layers. If the user does not choose a specific layer, the system allocates stock using FIFO. Sales items should only be marked packed, shipped, delivered, cancelled, or returned when the real operational event has occurred.

Figure A.5 shows the delivery-status controls used to keep the recorded sale state consistent with the actual order state.

A.6 Finance Follow-Up Workflow

Finance users use billing notes, payment batches, and credit notes to follow up completed or adjusted transactions. Billing notes group eligible customer sales for

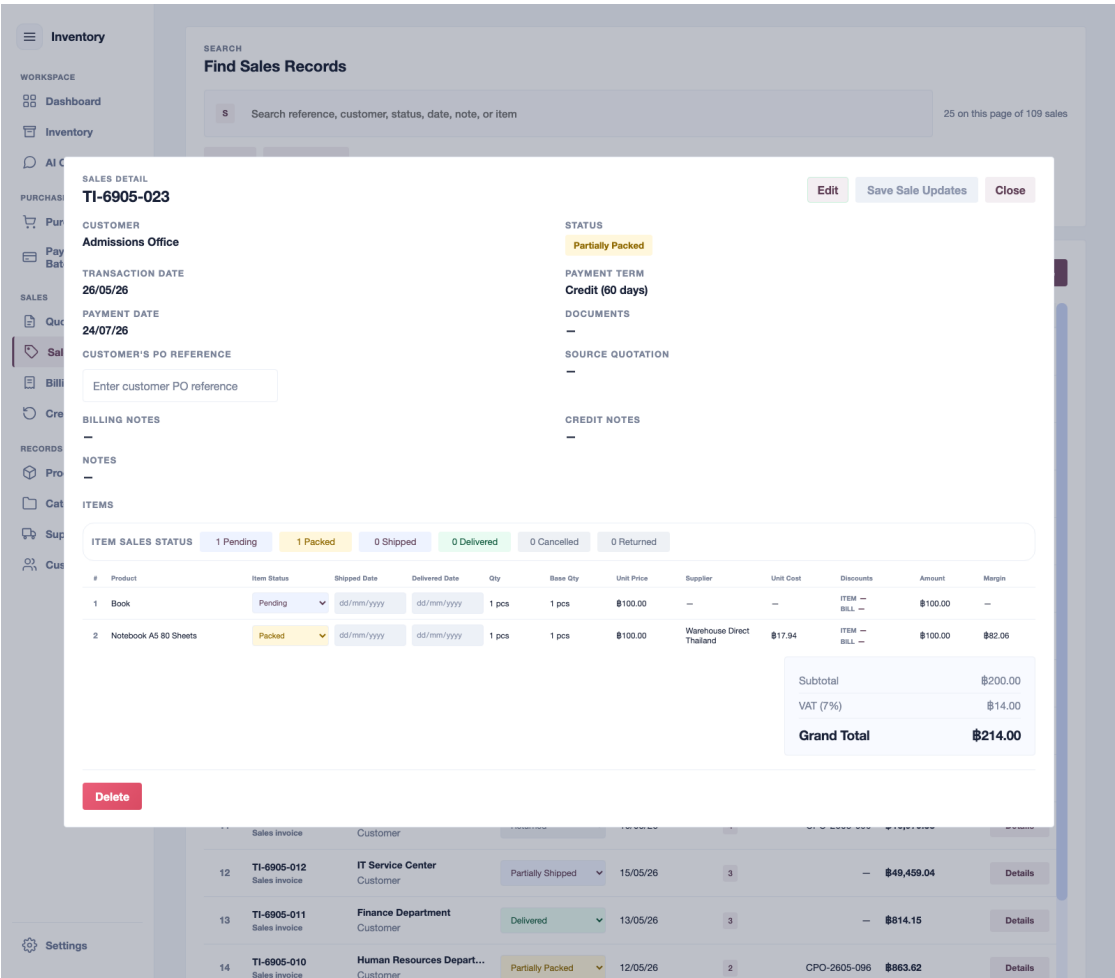


Figure A.5 Sales Detail With Delivery Status

collection. Payment batches group eligible supplier purchases for payment. Credit notes record returned or cancelled sale items.

Table A.2 explains the business side and main user action for each finance follow-up module.

Table A.2 Finance Follow-Up Modules

Module	Business Side	Main User Action
Billing Notes	Customer receivable	Select eligible shipped or delivered sales, issue the billing note, and record payment received.
Payment Batches	Supplier payable	Select eligible received purchases, schedule payment, and record the paid status and bank reference.
Credit Notes	Customer adjustment	Create credit notes from cancelled or returned sale items and review the tax and value adjustment.

These modules rely on backend eligibility checks. If a sale or purchase does not appear in the eligible list, the user should check its status, partner, and whether it has already been attached to another active document.

A.7 Inventory and Business Review

The dashboard and inventory pages are used for daily review. The dashboard summarizes urgent reorder items, stock movement, delivery planning, cash-flow pressure, popular products, and order coverage. The inventory page gives a more detailed stock-control view with product stock, pending purchases, demand, reorder values, supplier options, and product history.

Figure A.6 shows the dashboard overview used to review these business indicators together.

Users should review low-stock products before promising new delivery dates. They should also check delayed inbound purchases because a late supplier delivery can affect customer sales. Product history can be opened when a user needs to audit the purchase and sales records behind a stock value.

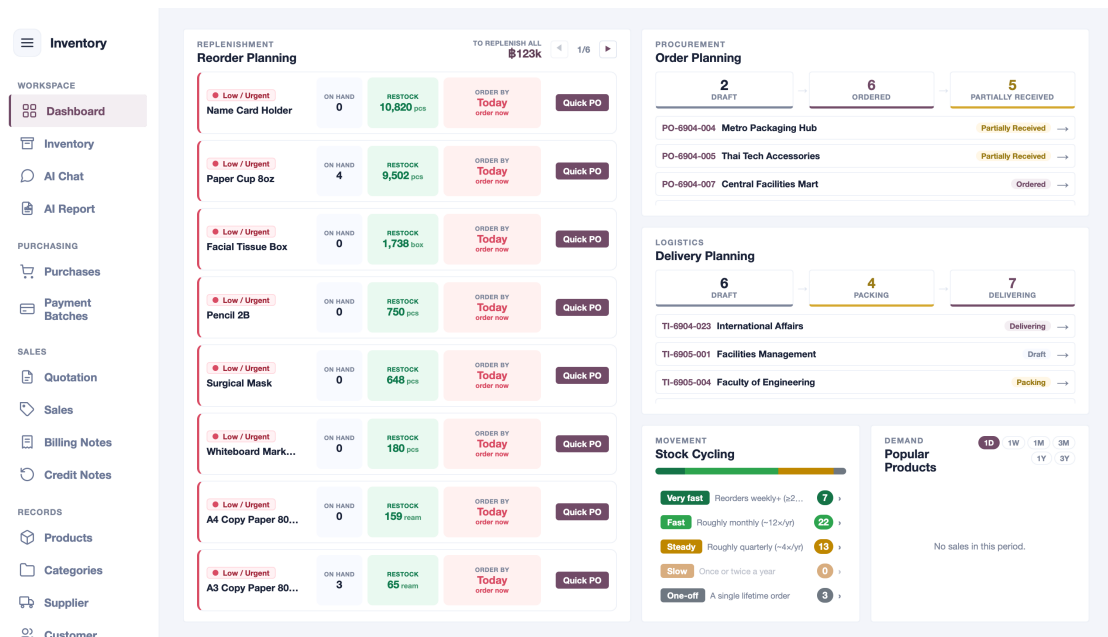


Figure A.6 Dashboard Overview for Business Review

A.8 AI Chat and AI Report

AI Chat is a read-only helper for asking supported operational questions. Suitable questions include low-stock checks, supplier or customer summaries, overdue receivables, overdue payables, order coverage, and document reference lookups. With an OpenAI API key configured, the answer includes AI-written wording, a conclusion, and question-specific highlights based on the verified backend presentation. Without a key or after an API failure, the page displays the deterministic local summary. In both modes, backend data and transaction records remain the source of truth.

AI Report creates printable reports for a selected supplier, customer, or product. The user selects the report scope, selects the record, chooses a reporting period if needed, and generates the report. The generated report includes backend-prepared metrics, related records, chart data, and a written analysis. The report can be printed or saved as a PDF from the browser.

A.9 Administration

Administrators use the User Access page to create staff accounts, assign roles, activate or deactivate users, and review role permissions. The Activity Log page is used to review important system activity such as login, creation, update, and deletion events.

Staff should be assigned only the access required for their responsibility.

A.10 Quick Reference

Table A.3 maps common user goals to the page and main action used to complete each goal.

Table A.3 Quick Reference for Common User Goals

Goal	Page	Main Action
Check business overview	Dashboard	Review stock, delivery, receivable, payable, and coverage summaries.
Check stock and reorder needs	Inventory	Filter stock rows, inspect reorder signals, and open product history.
Create product records	Products	Set SKU, category, base unit, unit conversions, suppliers, and reorder information.
Prepare a customer offer	Quotation	Add requested items, review stock sufficiency, and convert to purchase or sale when appropriate.
Buy from supplier	Purchases	Create purchase orders and mark items received only after goods arrive.
Fulfill customer order	Sales	Create sales, review FIFO allocation, and update delivery statuses.
Collect customer payment	Billing Notes	Group eligible sales and record received payment.
Pay supplier	Payment Batches	Group eligible purchases and record paid status.
Record returns or cancellations	Credit Notes	Issue credit notes from returned or cancelled sale items.
Ask operational questions	AI Chat	Ask supported read-only questions about stock, partners, AP, AR, and documents.
Generate printable analysis	AI Report	Select supplier, customer, or product and generate a printable report.
Manage staff access	User Access	Create users, assign roles, and review permissions.
Change interface language	Settings	Switch the user interface between English and Thai.